LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 4. Operators and Expressions

Lecture: 35. String Concatenation & Repetition

Section 1: Lecture Summary

This lecture covers **string concatenation and repetition** in Python, along with a brief review of arithmetic operators and their behavior across different data types. It starts with a table of arithmetic operators, then focuses mainly on how **addition (+)** and **multiplication ()** operators work with strings — concatenation and repetition respectively. The lecture then explores arithmetic operations on other Python data types such as integer, float, boolean, and complex, highlighting which operators are applicable to each. Finally, it discusses the **resultant data type** after performing arithmetic operations on mixed data types, explaining the type hierarchy: boolean < integer < float < complex.

Section 2: Key Concepts and Explanations

- **Revisiting Arithmetic Operators**

  Common arithmetic operators covered:

  - Addition (+)

  - Subtraction (-)

  - Multiplication ()

  - Division (/)

  - Floor Division (//)

  - Modulus (%)

  - Exponentiation (**)

- **Data Types in Python**

  - Integer (int)

- Float (float)

- Boolean (bool)

- Complex (complex)

- String (str)

- **String Concatenation and Repetition**

- Only two arithmetic operators work with strings:

- `+` for **concatenation** (joining two strings)

- `` for **repetition** (repeating a string multiple times)

- **Concatenation Details:**

- Both operands must be strings; concatenating a string with a non-string (e.g., int) causes a TypeError.

- Non-string values can be converted to strings using `str()` before concatenation.

- **Repetition Details:**

- Multiplying a string by an integer n repeats the string n times.

- Multiplying a string by a float (non-integer) raises a TypeError.

- **Arithmetic Operators with Other Data Types**

- **Integer:** All arithmetic operators work as expected.

- **Float:** All operators except floor division and modulus work with floats; floor division and modulus also work but result in float truncation and remainder respectively.

- **Boolean:** All arithmetic operators work with booleans, where `True` is treated as 1 and `False` as 0. Operations return integer results, not boolean.

- **Complex:** All operators work except floor division and modulus. Floor division and modulus are not defined for complex numbers.

- **Resultant Data Type of Expressions**

- When combining multiple data types in arithmetic expressions, the result's data type is promoted to the largest type among the operands, with the hierarchy:

```
    bool < int < float < complex
```

- Examples:

  - `True + True` returns `2` of type int

  - `True + 5` returns `6` of type int

  - `3 + 2.4` returns `5.4` of type float

  - `True + 2.4` returns `3.4` of type float

  - `3 + (2+3j)` returns `(5+3j)` of type complex

  - Mixing all types: boolean + integer + float + complex results in complex type

Section 3: Example Code and Use Cases

String Concatenation examples

Concatenating string with string

Trying to concatenate string with integer causes error

s + 10 TypeError: can only concatenate str (not "int") to str

Converting integer to string before concatenation

Concatenating two strings

```
s = 'abc'
print(s + '10')  # Output: 'abc10'


a = 10
print(s + str(a))  # Output: 'abc10'

s1 = 'abc'
s2 = 'def'
print(s1 + s2)  # Output: 'abcdef'
```

Explanation: Shows how addition operator combines strings only, and conversion from int to str is needed for safe concatenation.

String Repetition examples

Multiply by a non-integer raises error

s * 5.5 TypeError: can't multiply sequence by non-int of type 'float'

Repeating string with spaces

```
s = 'a'
print(s * 5)  # Output: 'aaaaa'


s = 'abc '
print(s * 5)  # Output: 'abc abc abc abc abc '
```

Explanation: Multiplying a string by an integer repeats it multiple times; float multiplication is not allowed.

Arithmetic operations with booleans

Arithmetic operations with floats (floor division and modulus)

```
print(True + True)  # Output: 2 (int)
print(True + 5)     # Output: 6 (int)
print(True * False) # Output: 0 (int)

print(10.5 // 3.1)  # Output: 3.0 (floor division truncates decimal part)
print(10.5 % 3.1)   # Output: 1.2  (remainder of division)
```

Explanation: Boolean arithmetic treats True as 1 and False as 0, results promote to int. Floor division with floats truncates decimal part.

Complex arithmetic operations

Floor division or modulus with complex numbers causes error

a // b TypeError

a % b TypeError

```
a = 3 + 2j
b = 1 + 1j
print(a + b)   # Output: (4+3j)
print(a * b)   # Output: (1+8j)
```

Explanation: Addition, multiplication, division work with complex numbers, but floor division and modulus are not supported.

Resultant data types demonstration

```
print(type(True + True))       # <class 'int'>
print(type(True + 5))          # <class 'int'>
print(type(3 + 2.4))           # <class 'float'>
print(type(True + 2.4))        # <class 'float'>
print(type(3 + 2 + 3j))        # <class 'complex'>
print(type(True + 3 + 4.5 + (2 + 3j)))  # <class 'complex'>
```

Explanation: Demonstrates the promotion of operand types in mixed arithmetic expressions.

Section 4: Key Takeaways

- Only **addition (+)** and **multiplication ()** operators work with strings: addition concatenates, multiplication repeats.

- The operands in string concatenation must both be strings; convert non-string types using `str()`.

- Multiplying a string by a **non-integer** (e.g., float) raises a `TypeError`.

- All arithmetic operators work on integers and floats, with floor division (`//`) truncating the decimal quotient for floats.

- Booleans behave like integers in arithmetic operations (`True` = 1, `False` = 0), with results as integers.

- Complex numbers support addition, subtraction, multiplication, division, and exponentiation; floor division and modulus are unsupported.

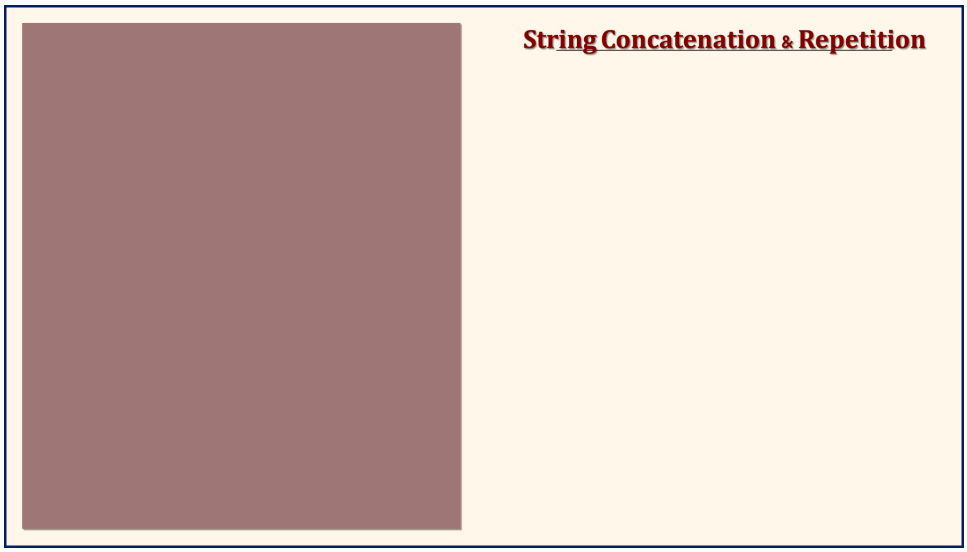
- The resultant type of mixed-type arithmetic expressions follows this hierarchy:

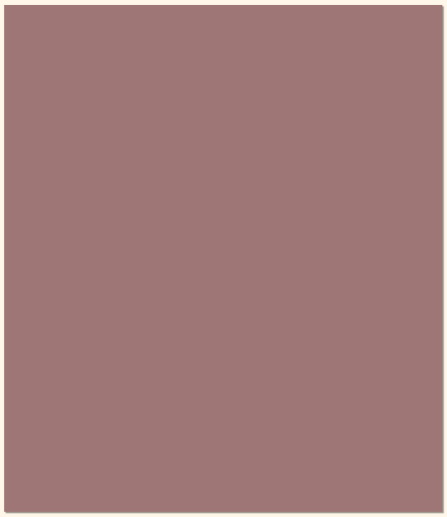
  `bool < int < float < complex` — the result is coerced to the highest type involved.

- Understanding these behaviors helps prevent common type errors and is useful for certification or interview preparation.

Lecture in Slides

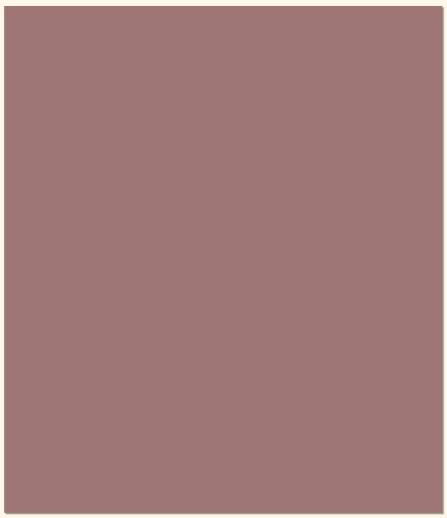
Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.





String Concatenation & Repetition

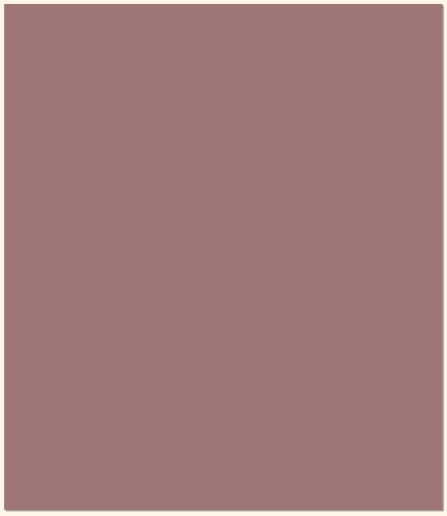
Revisiting Arithmetic Operator



String Concatenation & Repetition

Revisiting Arithmetic Operator

String Concatenation

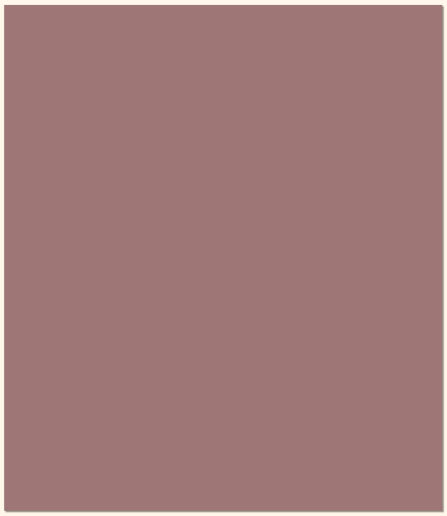


String Concatenation & Repetition

Revisiting Arithmetic Operator

String Concatenation

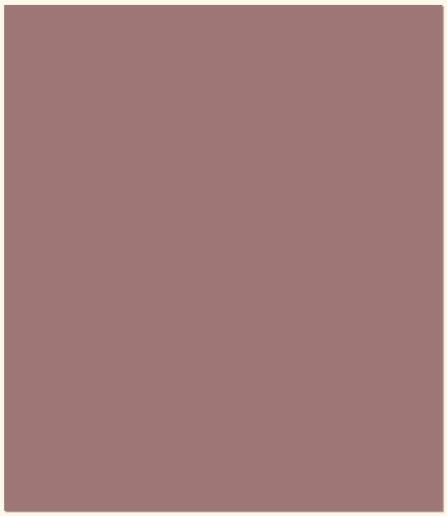
String Repetition



String Concatenation & Repetition

Revisiting Arithmetic Operator

String Concatenation & Repetition

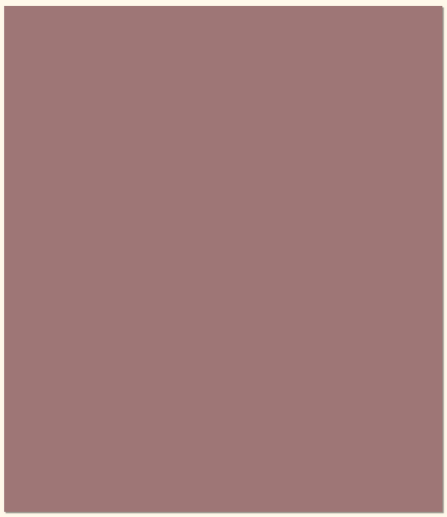


String Concatenation & Repetition

Revisiting Arithmetic Operator

String Concatenation & Repetition

Arithmetic with All Datatypes



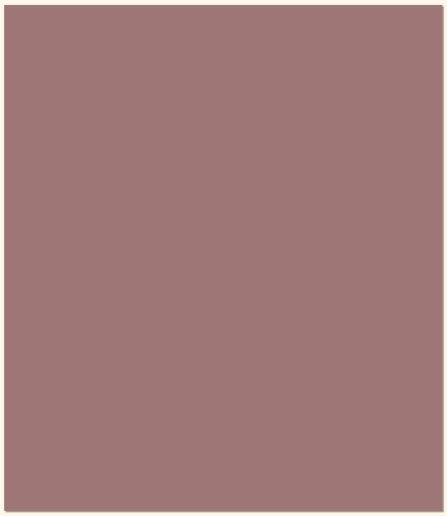
String Concatenation & Repetition

Revisiting Arithmetic Operator

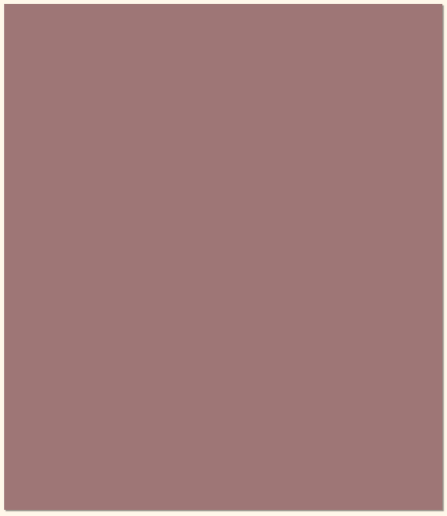
String Concatenation & Repetition

Arithmetic with All Datatypes

Resultant Datatype of Expression

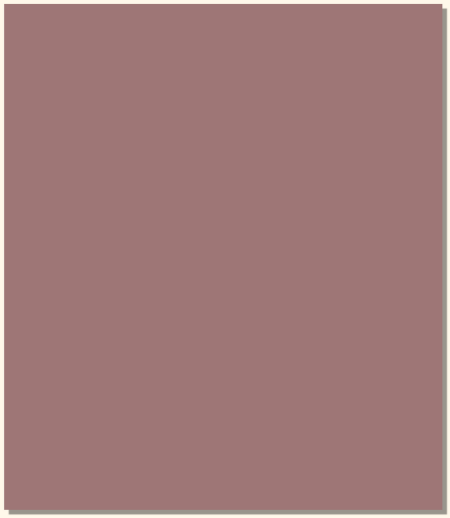


Revisiting Arithmetic Operator



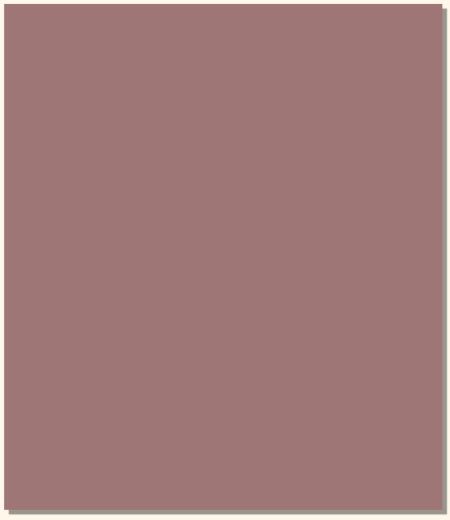
Arithmetic Operators

Arithmetic Operators

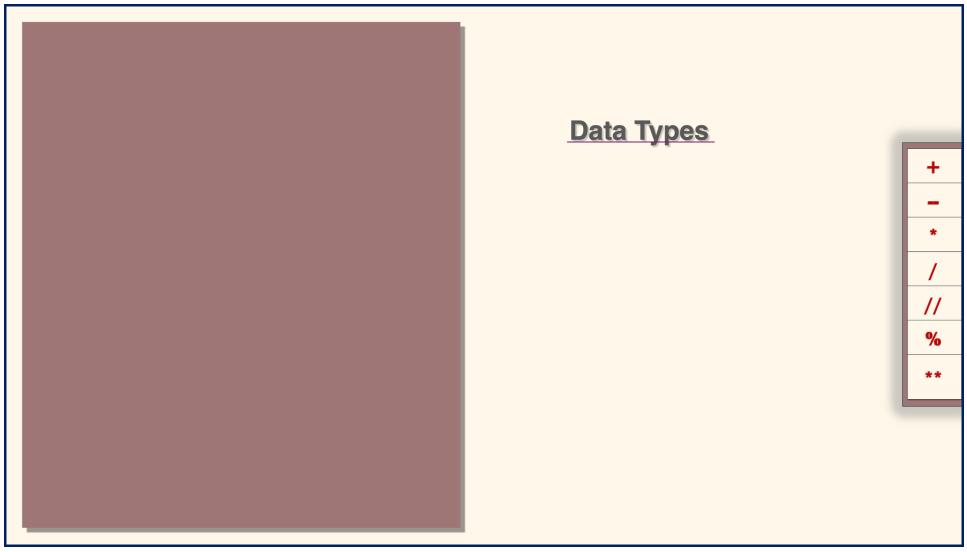


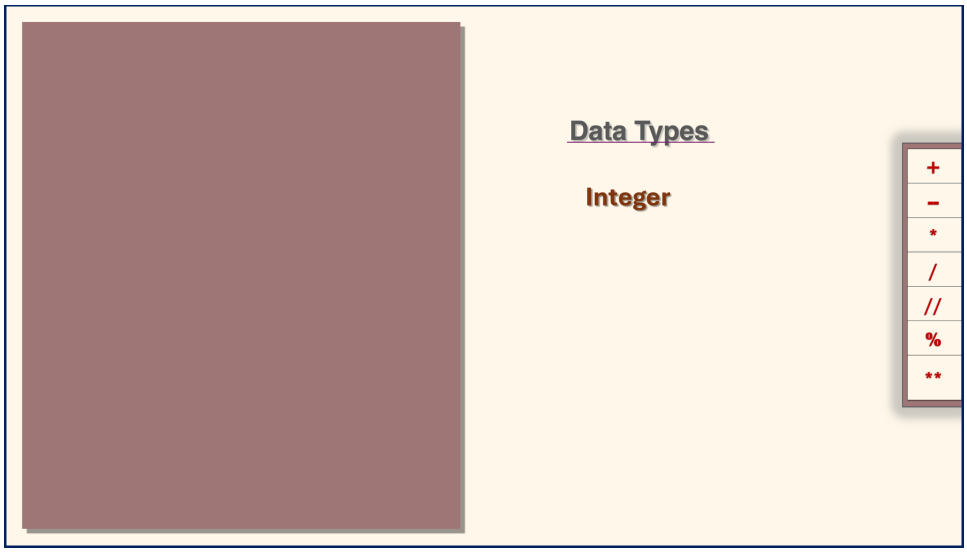
Arithmetic Operators	
+	Addition
-	Subtraction
*	Multiplication
/	Division (float result)
//	Floor Division
%	Modulus (remainder)
**	Exponentiation (power)

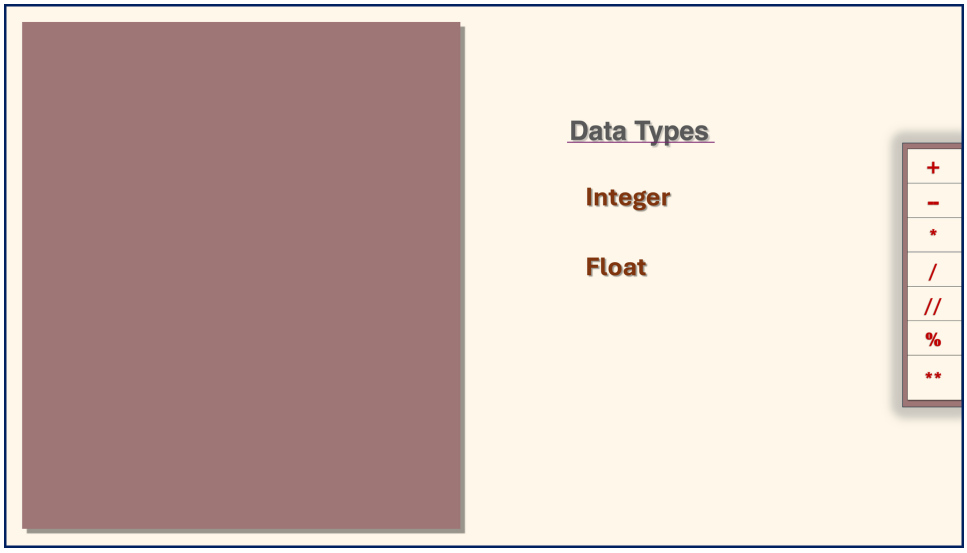
Arithmetic Operators

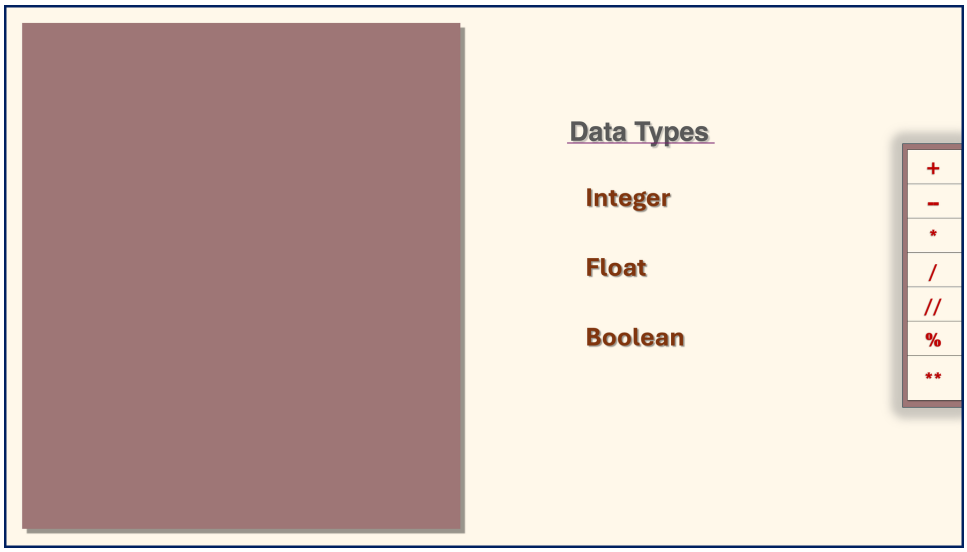


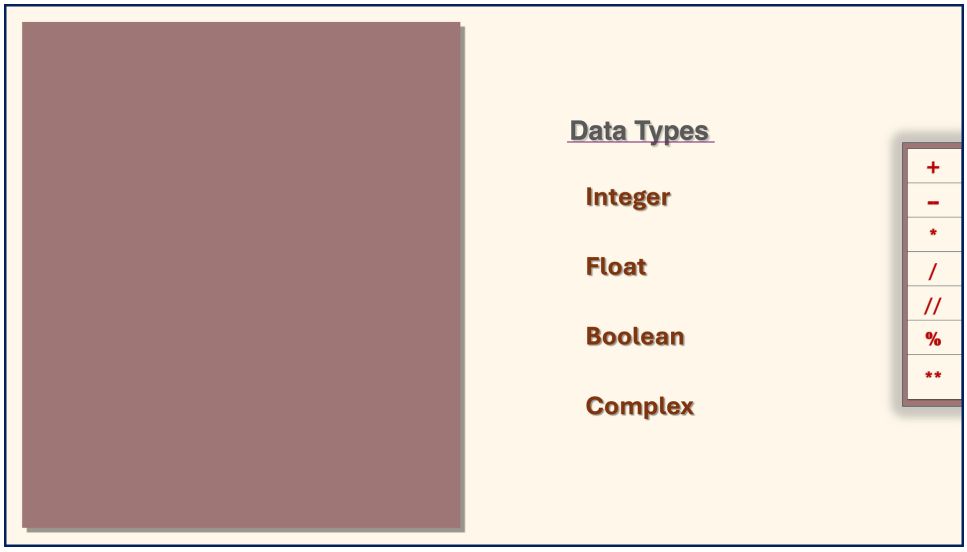
+
-
*
/
//
%
**

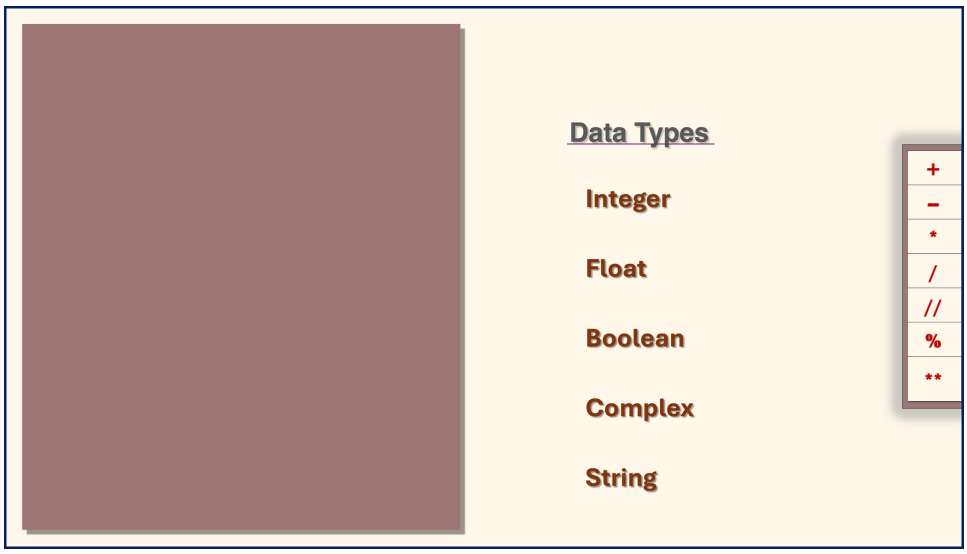

Data Types
+
-
*
/
//
%
**

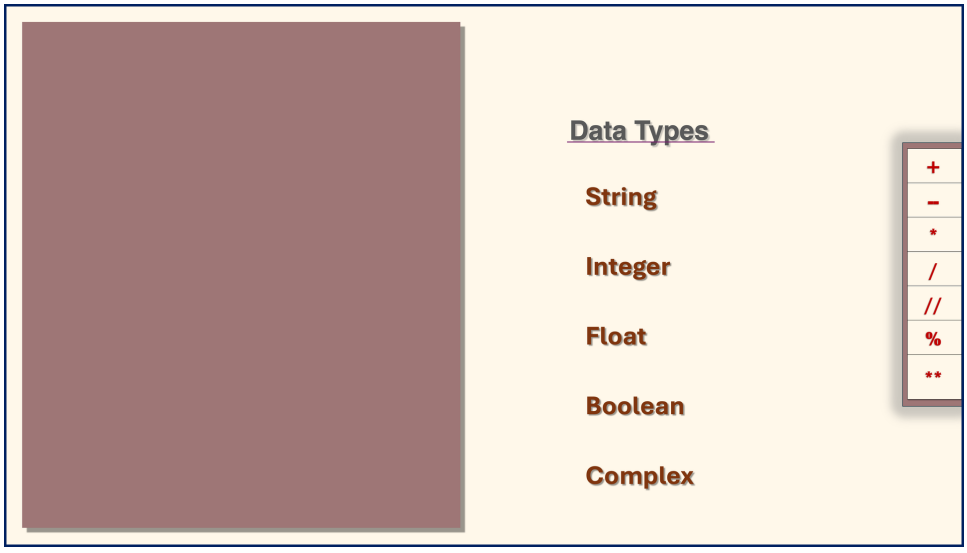

Data Types
Integer
+
-
*
/
//
%
**

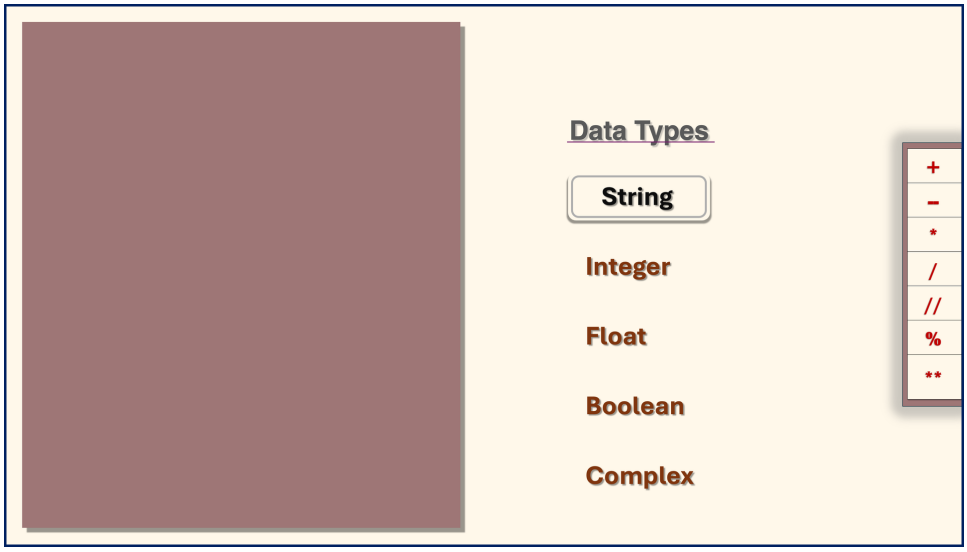

Data Types
Integer
Float
+
-
*
/
//
%
**

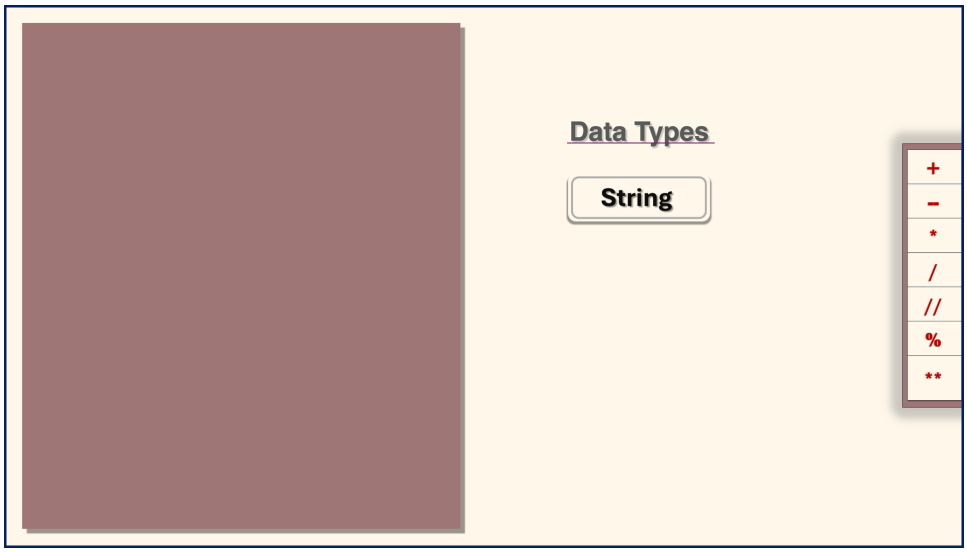

Data Types
Integer
Float
Boolean
+
-
*
/
//
%
**

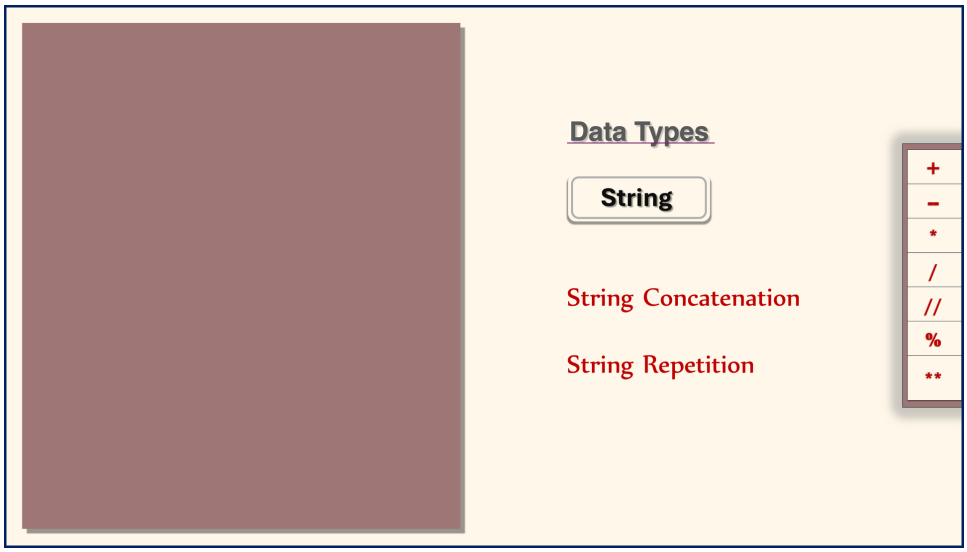

Data Types
Integer
Float
Boolean
Complex
+
-
*
/
//
%
**

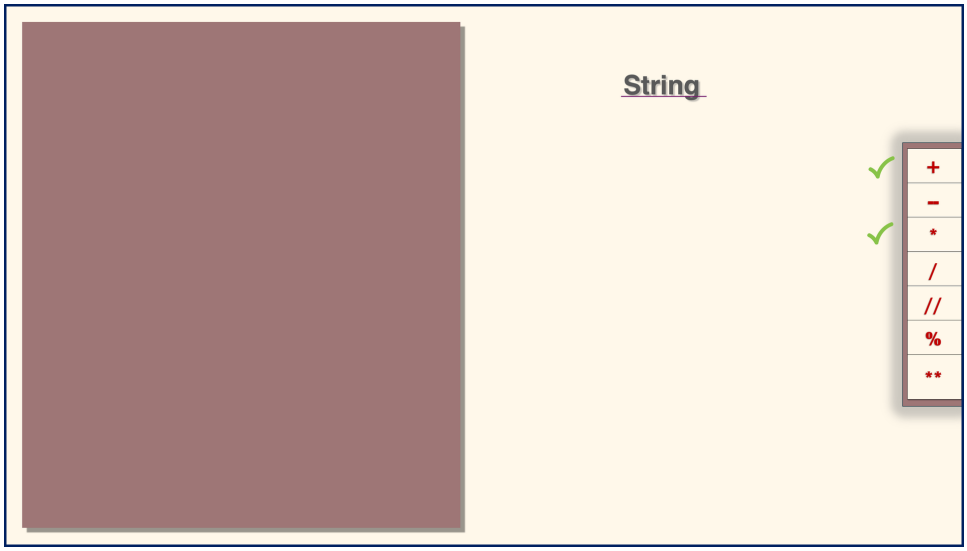

Data Types
Integer
Float
Boolean
Complex
String
+
-
*
/
//
%
**

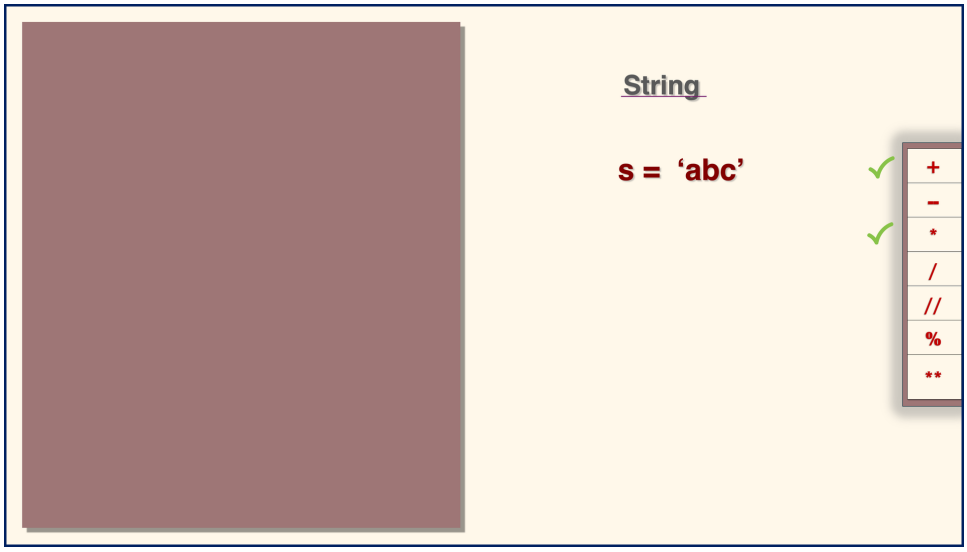

Data Types
String
Integer
Float
Boolean
Complex
+
-
*
/
//
%
**

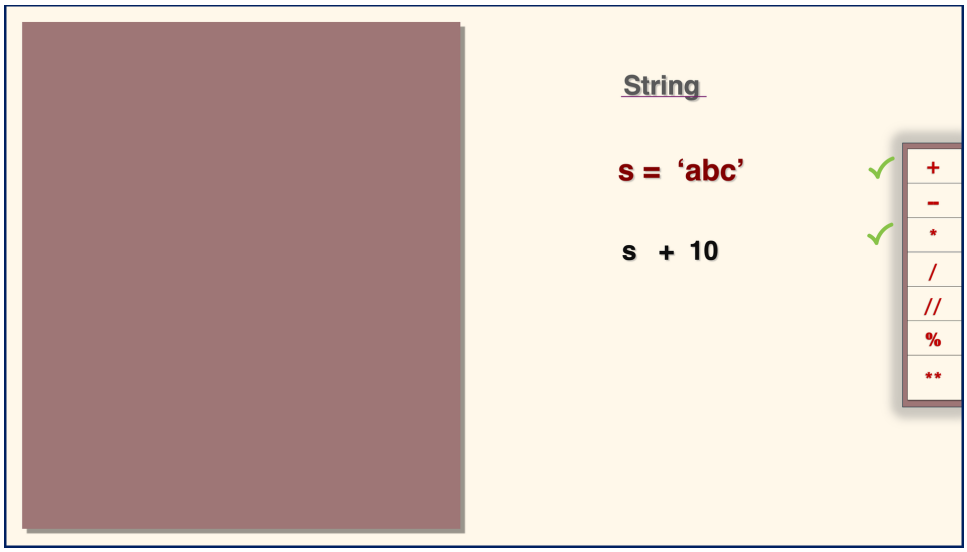

Data Types
String
Integer
Float
Boolean
Complex
+
-
*
/
//
%
**

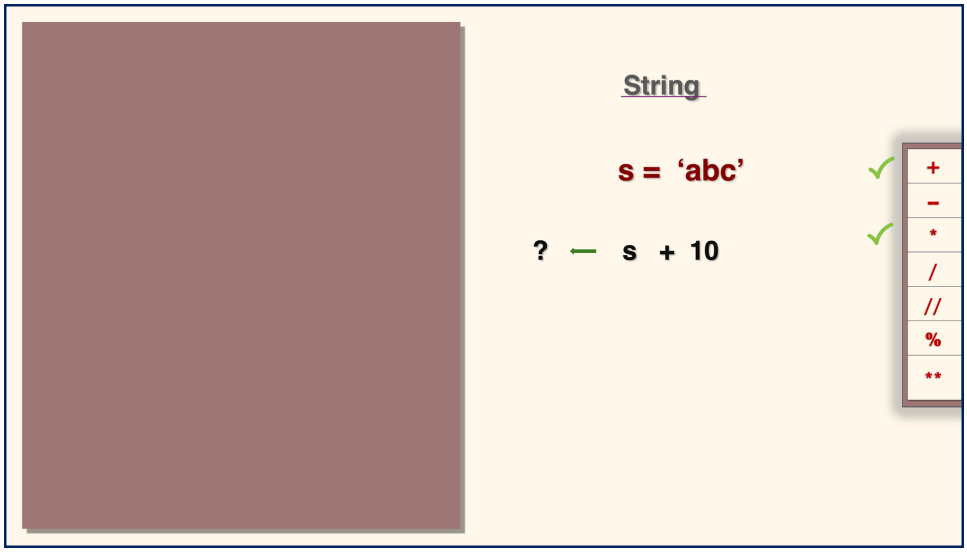

Data Types
String
+
-
*
/
//
%
**

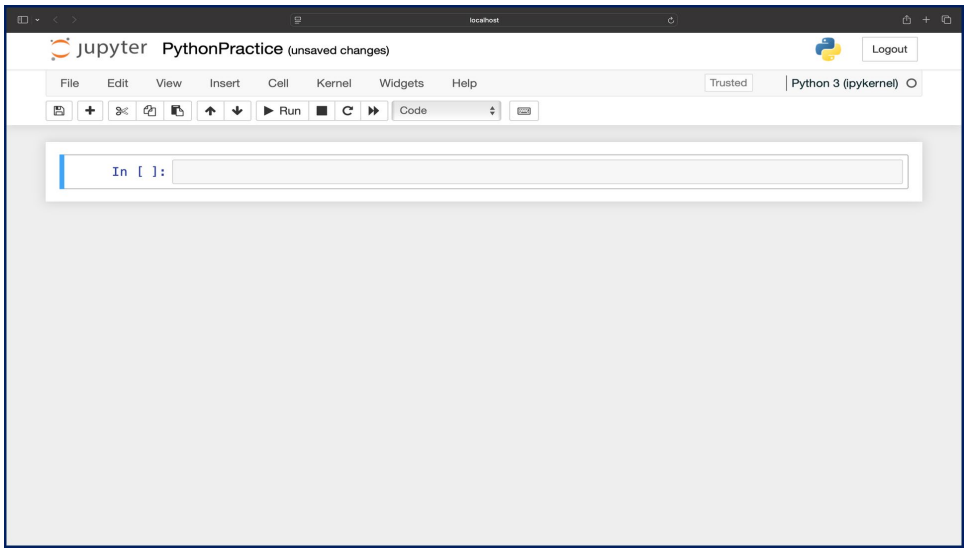

Data Types
String
String Concatenation
String Repetition
+
-
*
/
//
%
**

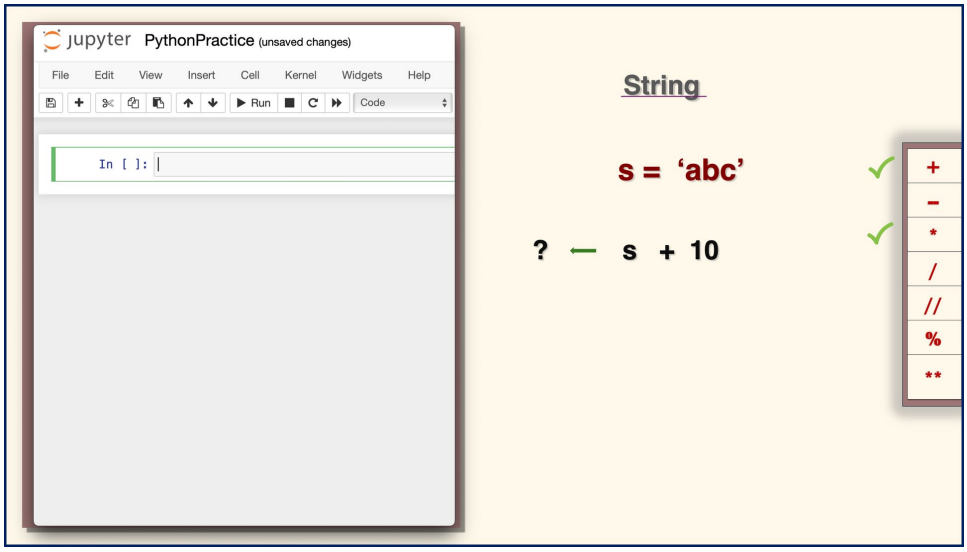

String
+
-
*
/
//
%
**

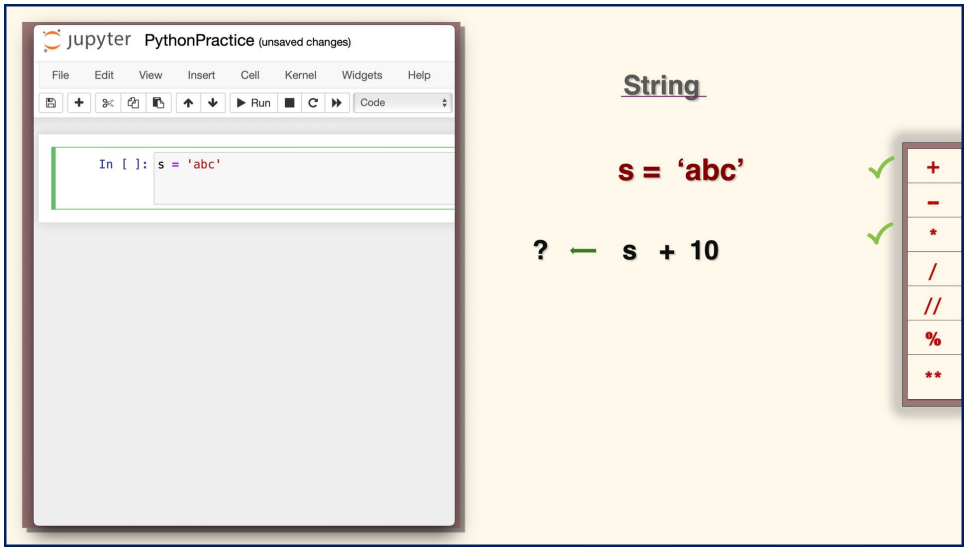

String
s = ‘abc’
+
-
*
/
//
%
**

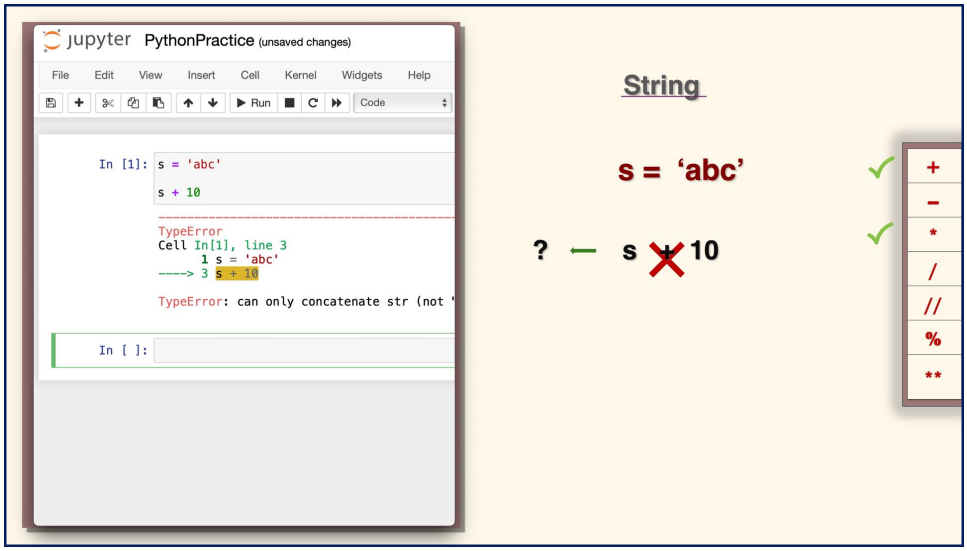

String
s = ‘abc’
s + 10
+
-
*
/
//
%
**

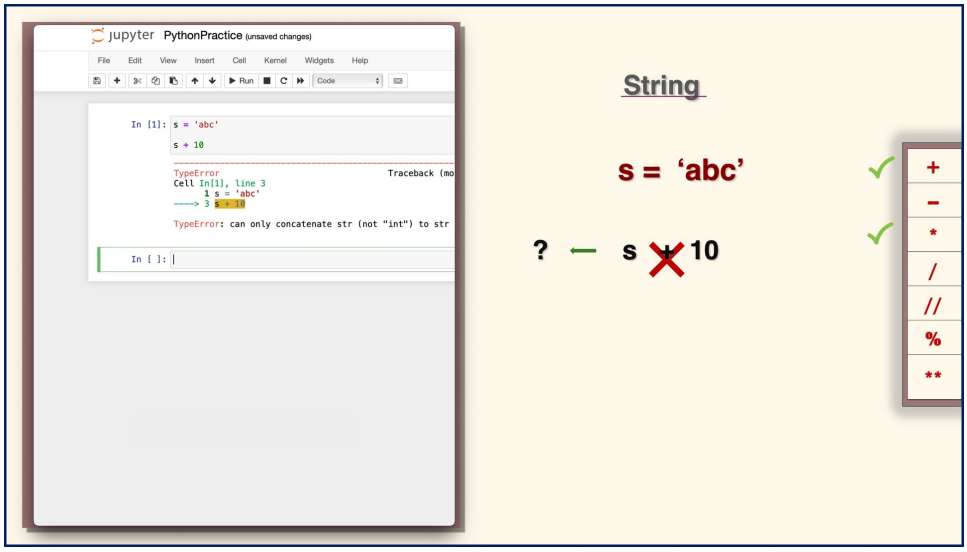

String
s = ‘abc’
? ← s + 10
+
-
*
/
//
%
**

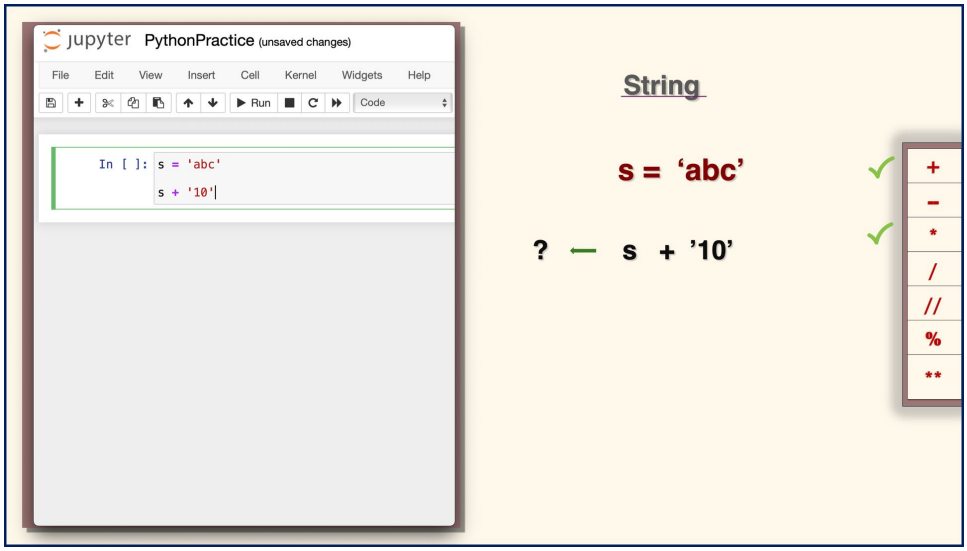

localhost
jupyter PythonPractice (unsaved changes)
Logout
File Edit View Insert Cell Kernel Widgets Help
Trusted
Python 3 (ipykernel)
Run
Code
In [ ]:

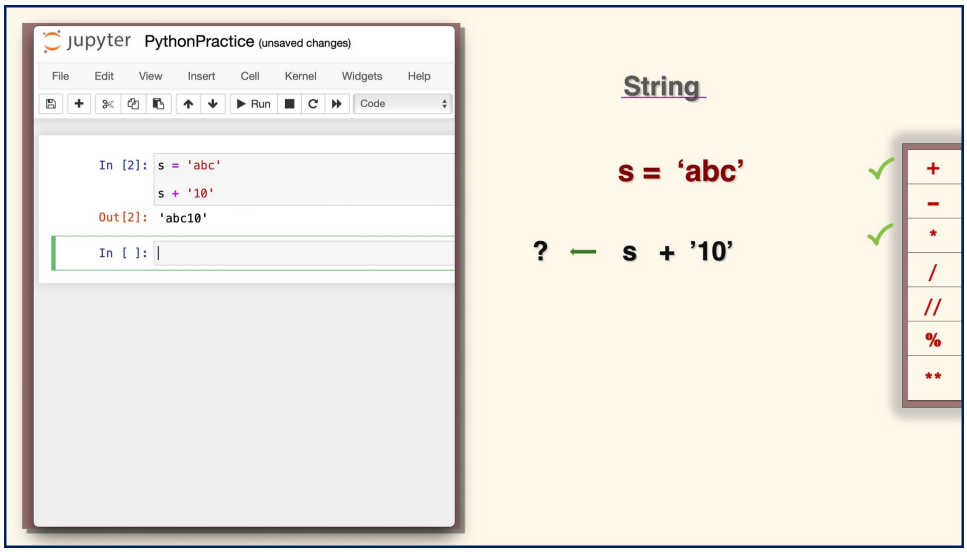

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [ ]:
String
s = 'abc'
? ← s + 10
+
-
*
/
//
%
**

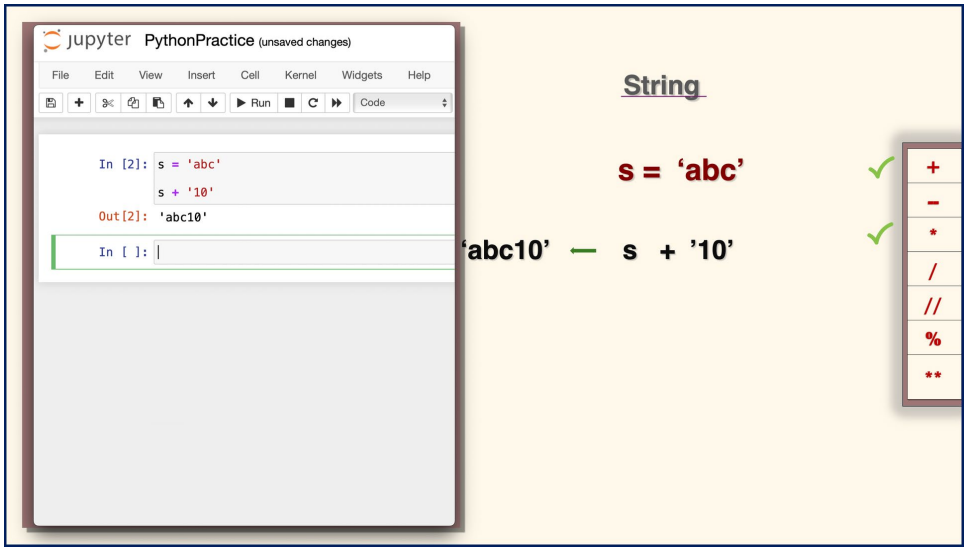

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]: s = 'abc'
String
s = 'abc'
? ← s + 10
+
-
*
/
//
%
**

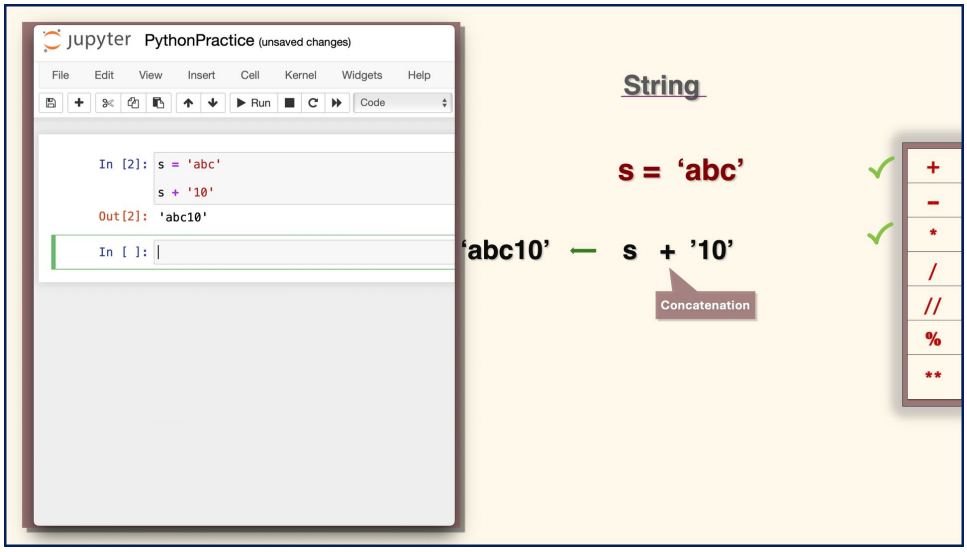

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [1]: s = 'abc'
s + 10
TypeError
Cell In[1], line 3
1 s = 'abc'
----> 3 s + 10
TypeError: can only concatenate str (not
In [ ]:
String
s = 'abc'
? ← s + 10
+
-
*
/
//
%
**

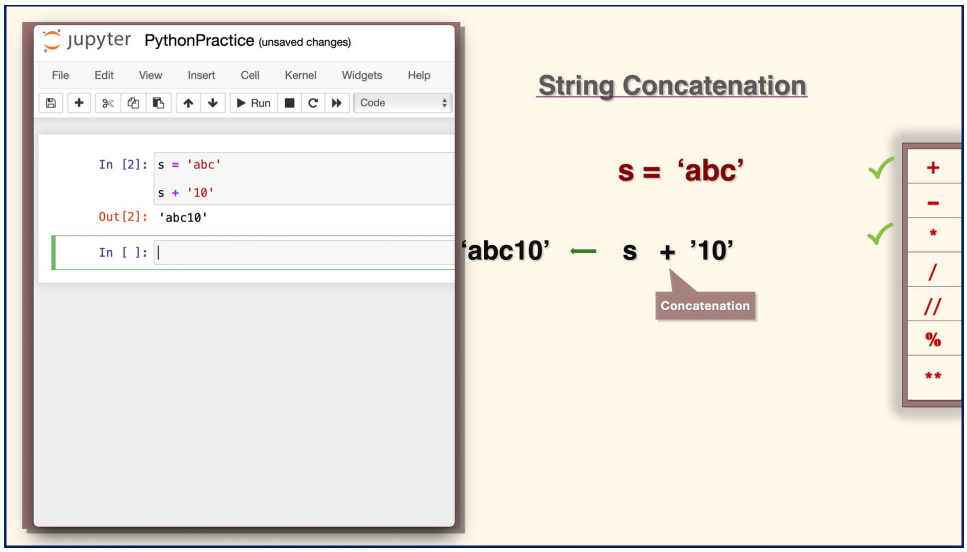

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [1]: s = 'abc'
s + 10
TypeError Traceback (mo
Cell In[1], line 3
1 s = 'abc'
----> 3 s + 10
TypeError: can only concatenate str (not "int") to str
In [ ]:
String
s = 'abc'
? ← s + 10
+
-
*
/
//
%
**

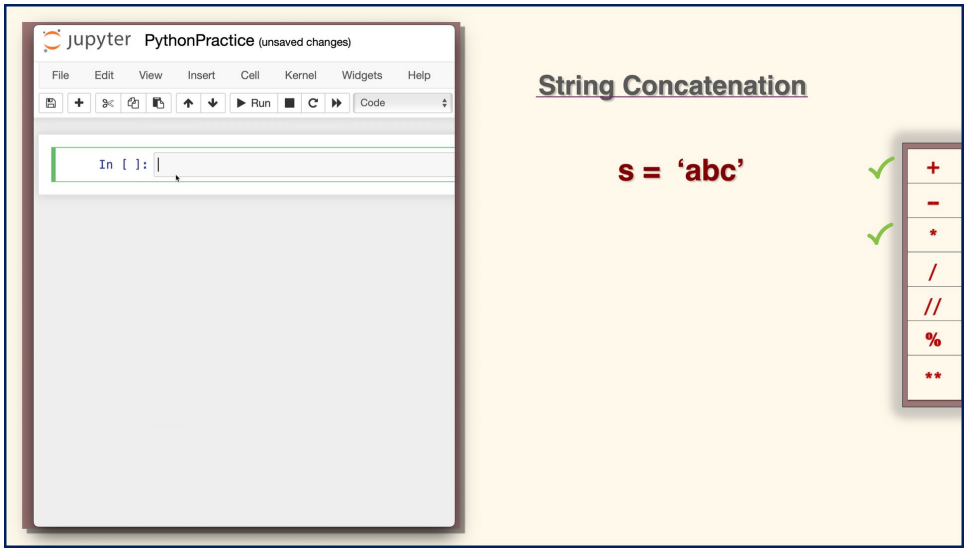

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]: s = 'abc'
s + '10'
String
s = 'abc'
? ← s + '10'
+
-
*
/
//
%
**

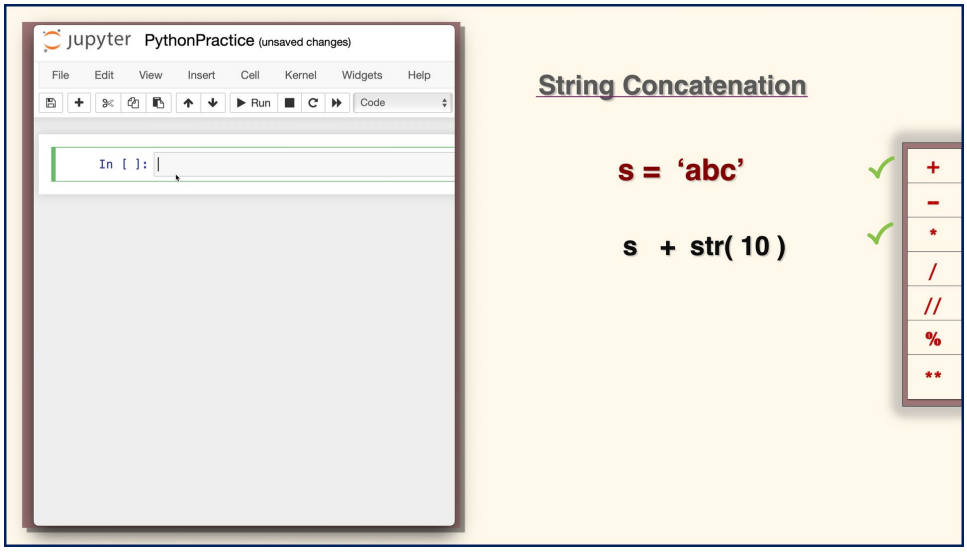

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [2]: s = 'abc'
s + '10'
Out[2]: 'abc10'
In [ ]:
String
s = 'abc'
? ← s + '10'
+
-
*
/
//
%
**

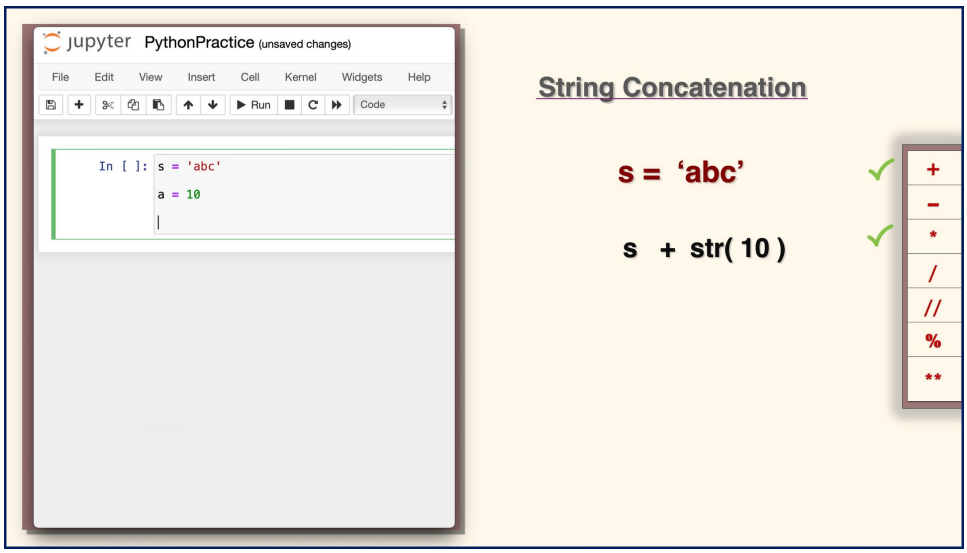

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [2]: s = 'abc'
s + '10'
Out[2]: 'abc10'
In [ ]:
String
s = 'abc'
'abc10' ← s + '10'
+
-
*
/
//
%
**

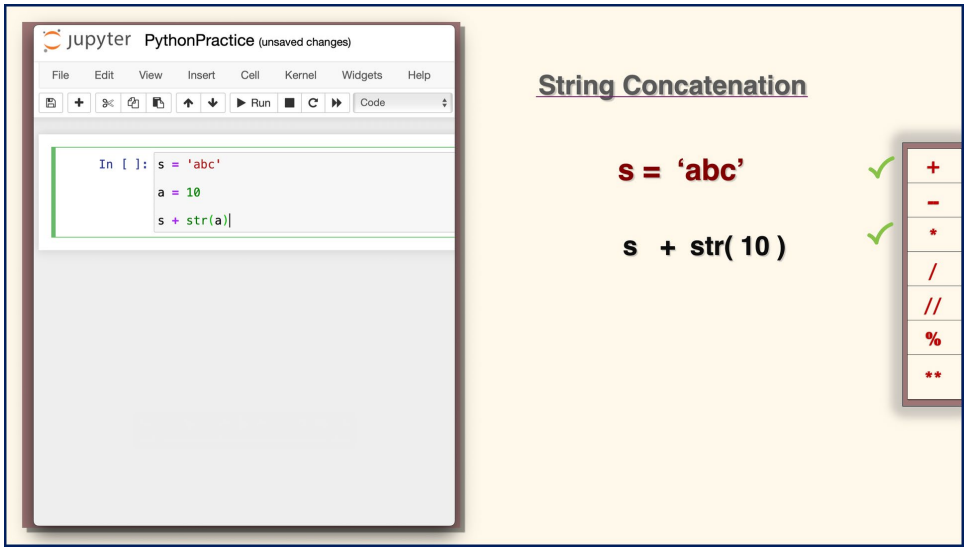

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [2]: s = 'abc'
s + '10'
Out[2]: 'abc10'
In [ ]:
String
s = ‘abc’
‘abc10’ ← s + ’10’
Concatenation
+
-
*
/
//
%
**

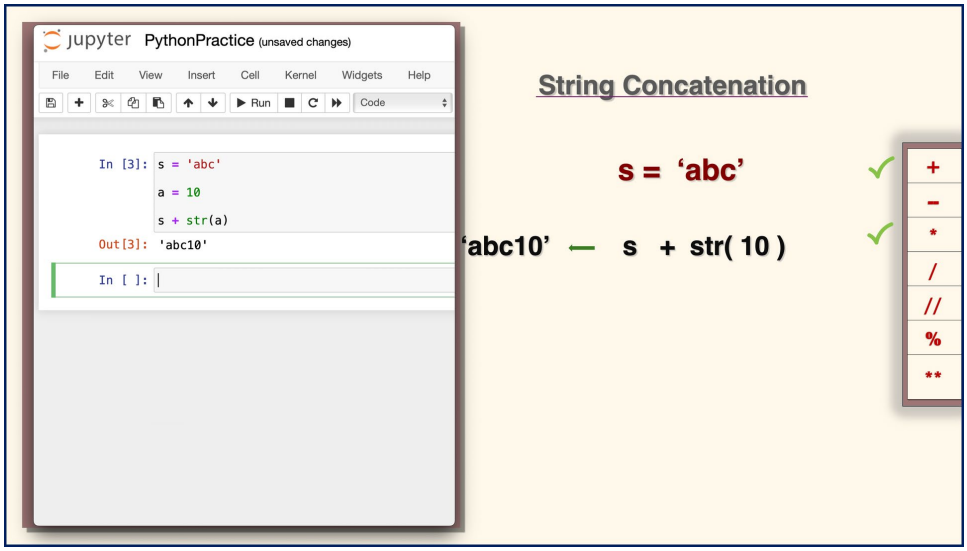

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [2]: s = 'abc'
s + '10'
Out[2]: 'abc10'
In [ ]:
String Concatenation
s = ‘abc’
‘abc10’ ← s + ’10’
Concatenation
+
-
*
/
//
%
**

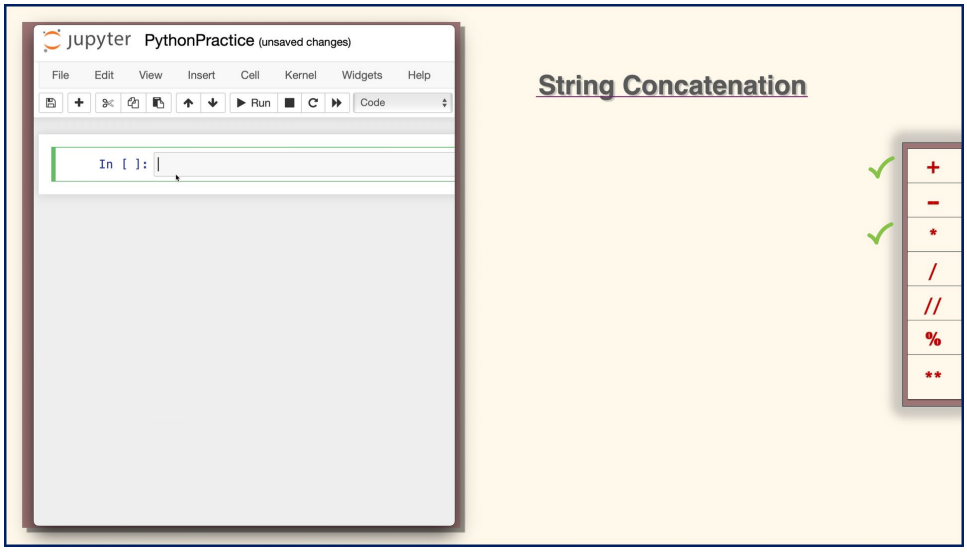

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]:
String Concatenation
s = ‘abc’
+
-
*
/
//
%
**

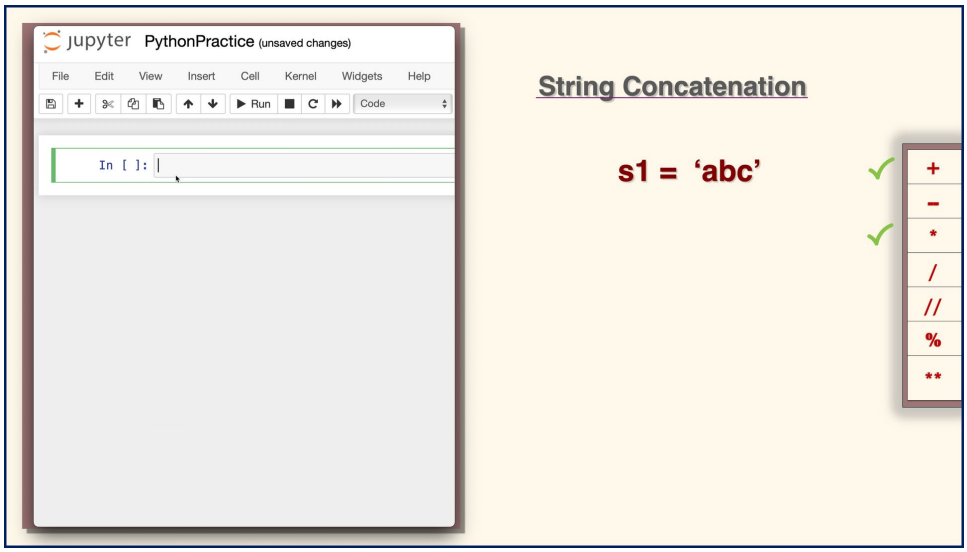

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]:
String Concatenation
s = ‘abc’
s + str(10)
+
-
*
/
//
%
**

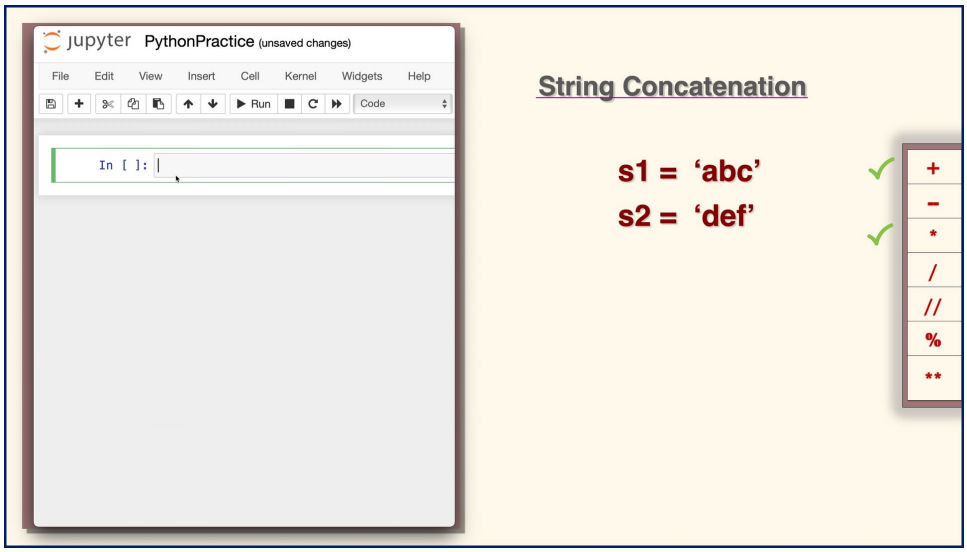

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]:
s = 'abc'
a = 10
String Concatenation
s = ‘abc’
s + str(10)
+
-
*
/
//
%
**

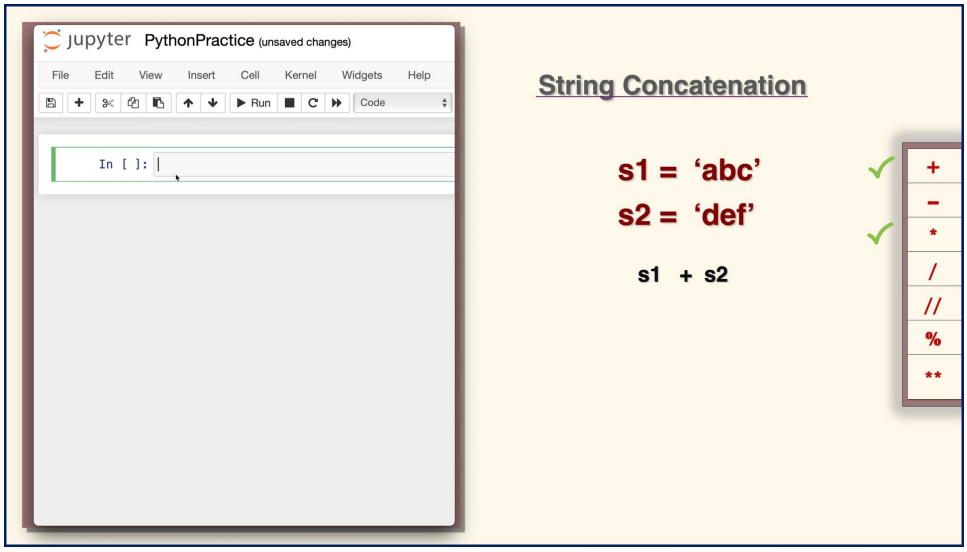

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]: s = 'abc'
a = 10
s + str(a)
String Concatenation
s = 'abc'
s + str(10)
+
-
*
/
//
%
**

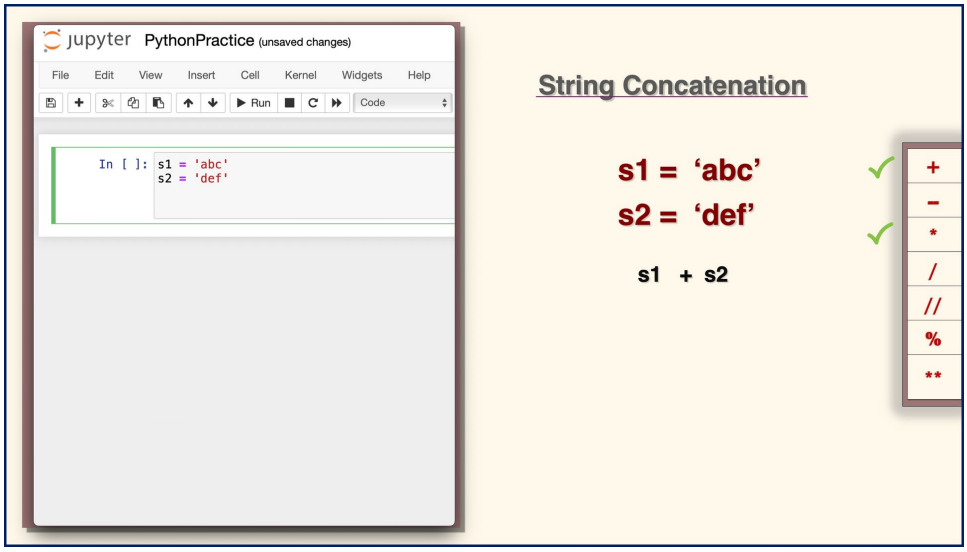

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [3]: s = 'abc'
a = 10
s + str(a)
Out[3]: 'abc10'
In [ ]:
String Concatenation
s = 'abc'
'abc10' ← s + str(10)
+
-
*
/
//
%
**

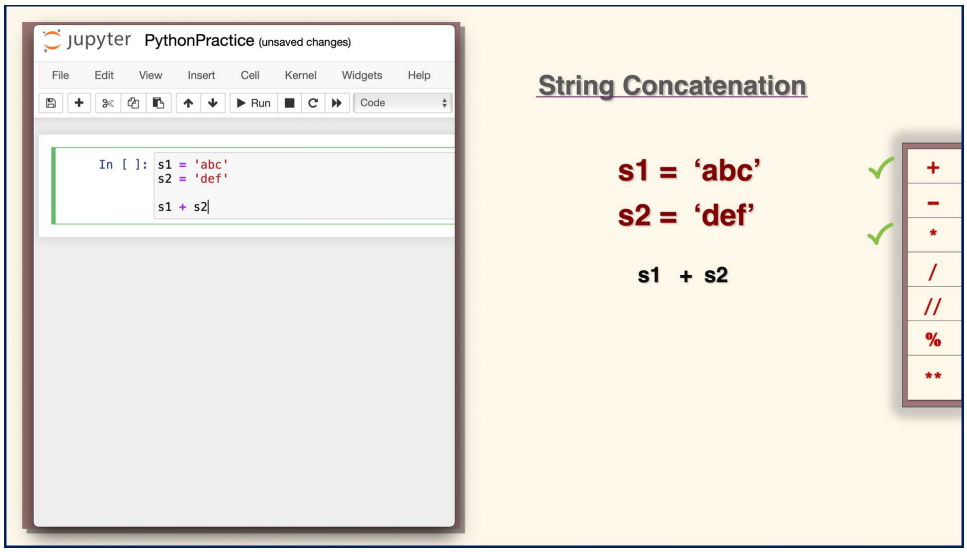

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]:
String Concatenation
+
-
*
/
//
%
**

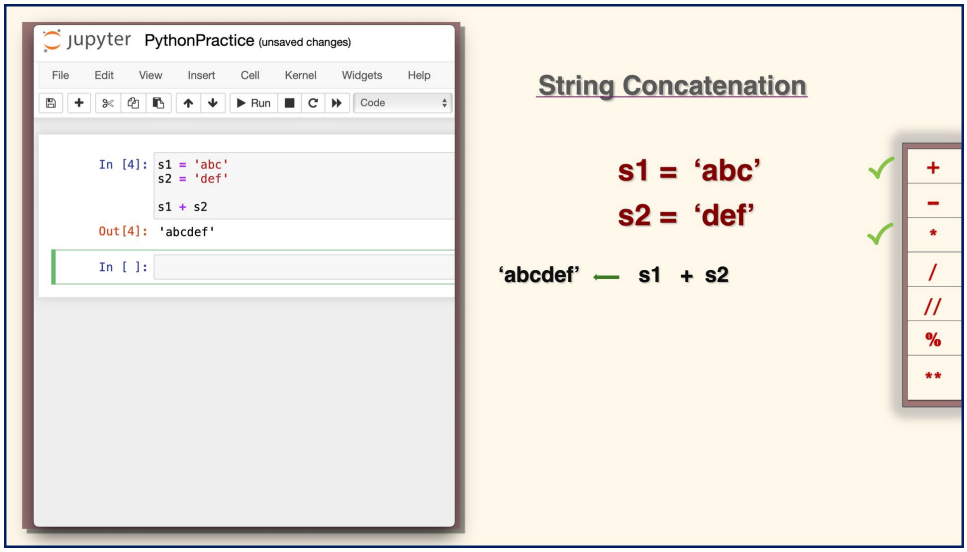

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]:
String Concatenation
s1 = 'abc'
+
-
*
/
//
%
**

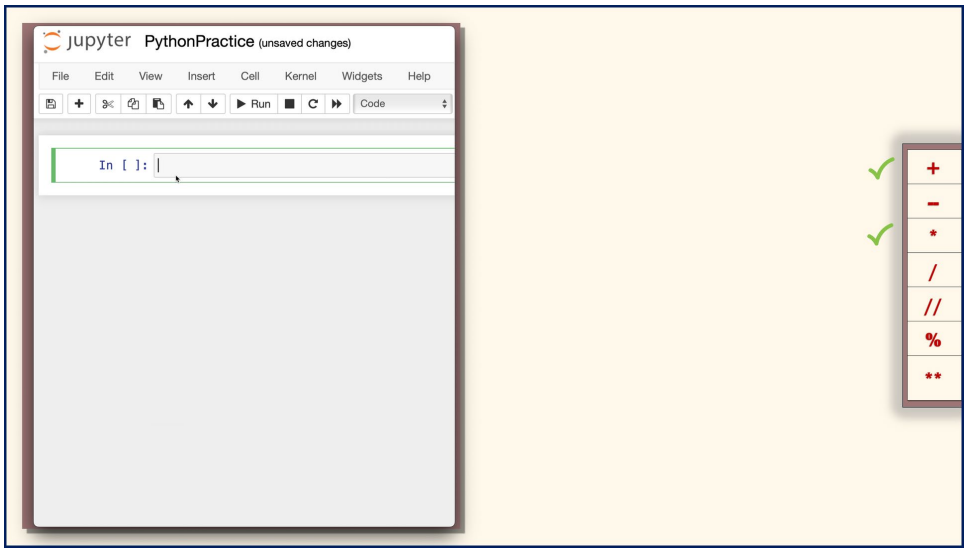

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]:
String Concatenation
s1 = 'abc'
s2 = 'def'
+
-
*
/
//
%
**


jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]:
String Concatenation
s1 = 'abc'
s2 = 'def'
s1 + s2
+
-
*
/
//
%
**

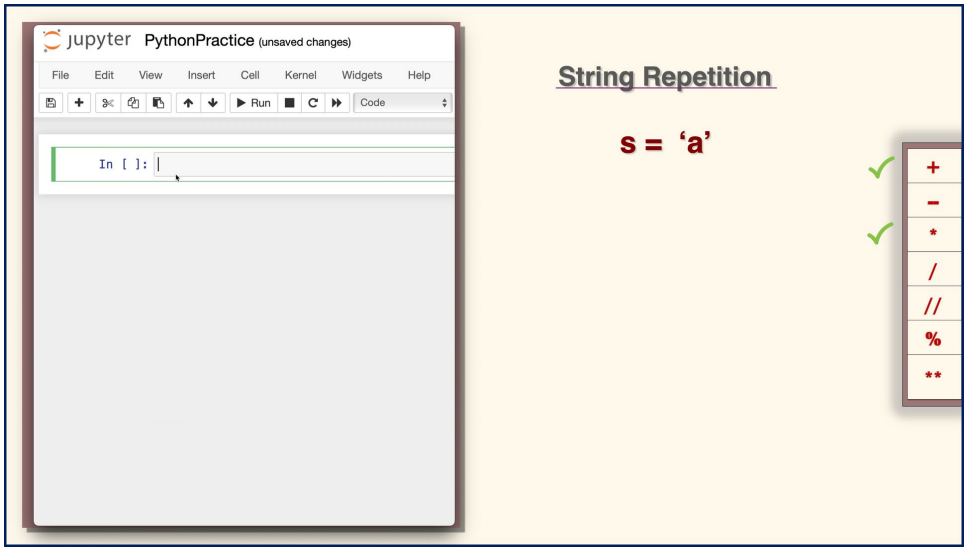

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [ ]:
s1 = 'abc'
s2 = 'def'
String Concatenation
s1 = ‘abc’
s2 = ‘def’
s1 + s2
+
-
*
/
//
%
**

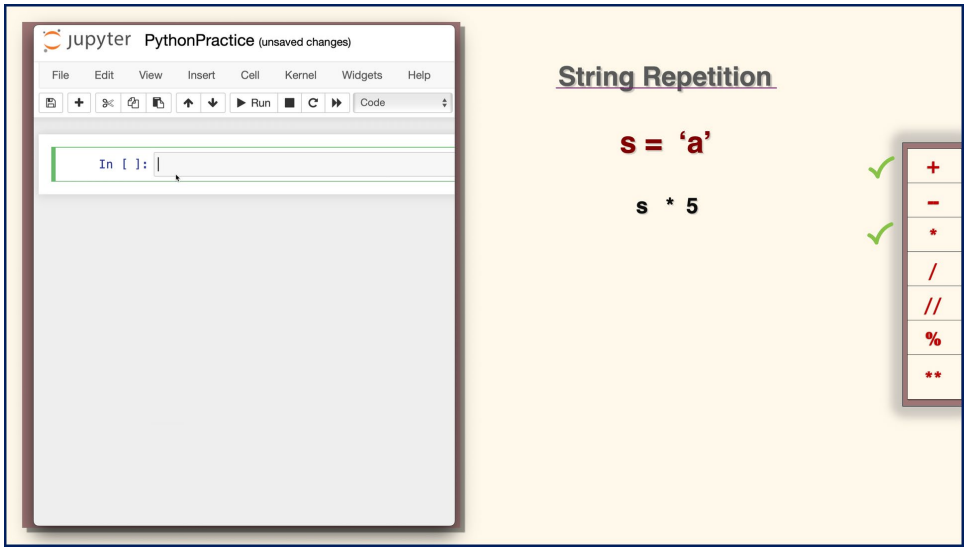

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]:
s1 = 'abc'
s2 = 'def'
s1 + s2
String Concatenation
s1 = ‘abc’
s2 = ‘def’
s1 + s2
+
-
*
/
//
%
**

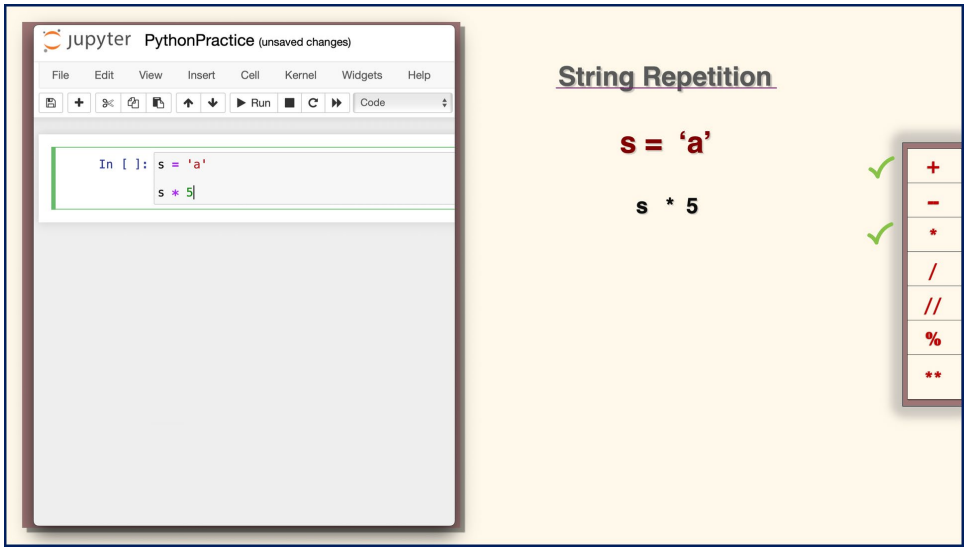

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [4]:
s1 = 'abc'
s2 = 'def'
s1 + s2
Out[4]: 'abcdef'
In [ ]:
String Concatenation
s1 = ‘abc’
s2 = ‘def’
‘abcdef’ ⟵ s1 + s2
+
-
*
/
//
%
**

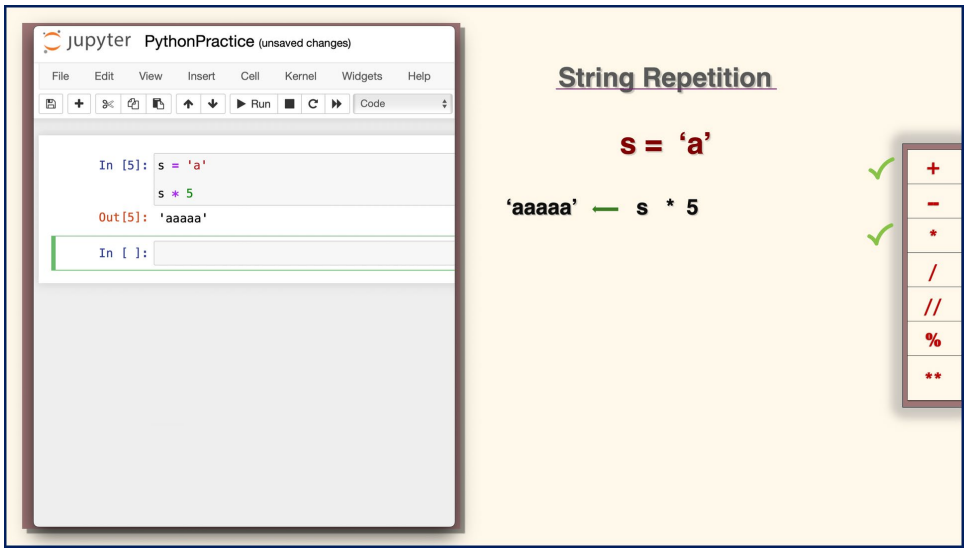

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [ ]:
+
-
*
/
//
%
**

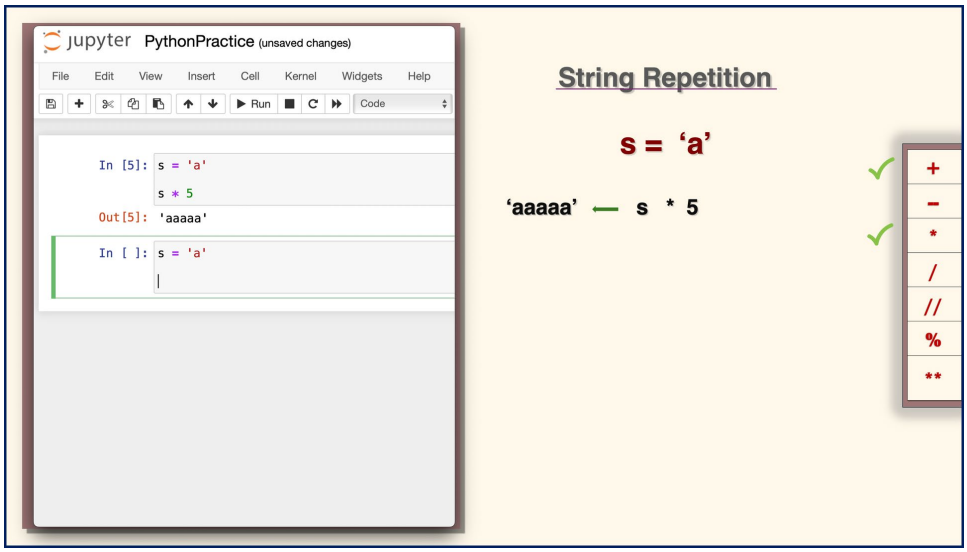

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [ ]:
String Repetition
+
-
*
/
//
%
**

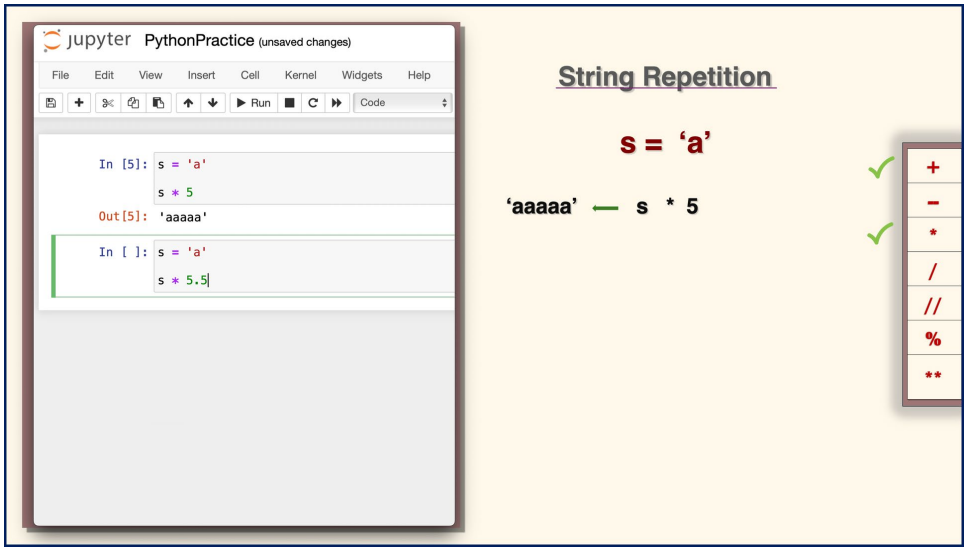

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]:
String Repetition
s = 'a'
+
-
*
/
//
%
**

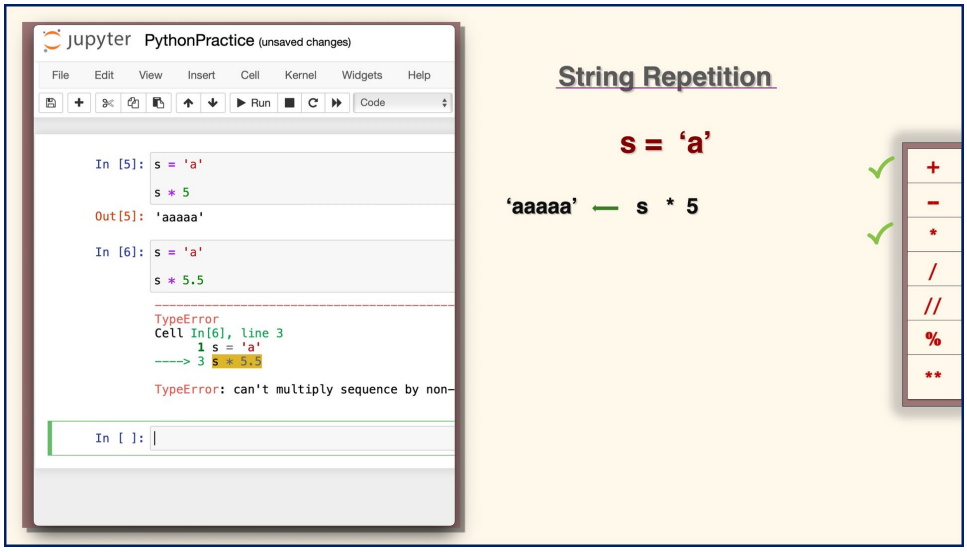

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]:
String Repetition
s = 'a'
s * 5
+
-
*
/
//
%
**

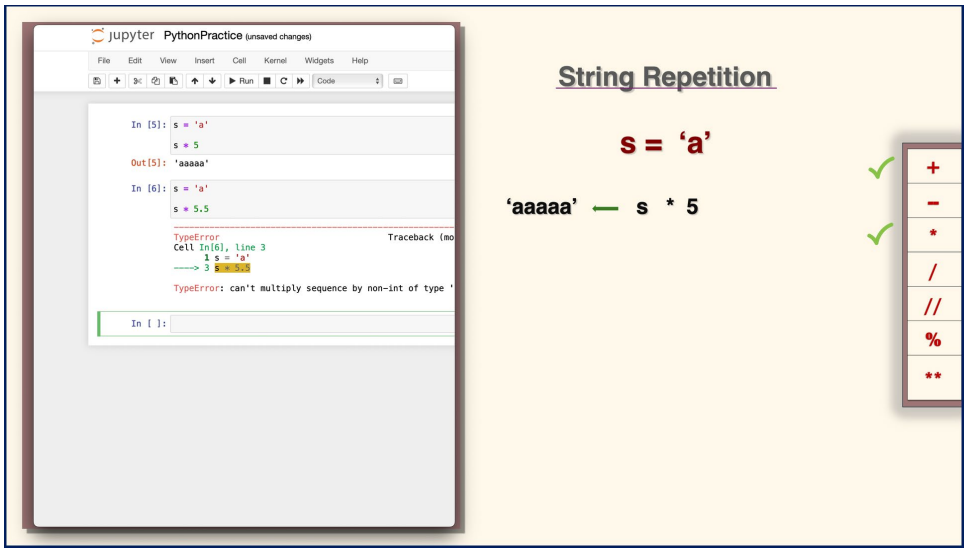

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]: s = 'a'
s * 5
String Repetition
s = ‘a’
s * 5
+
-
*
/
//
%
**

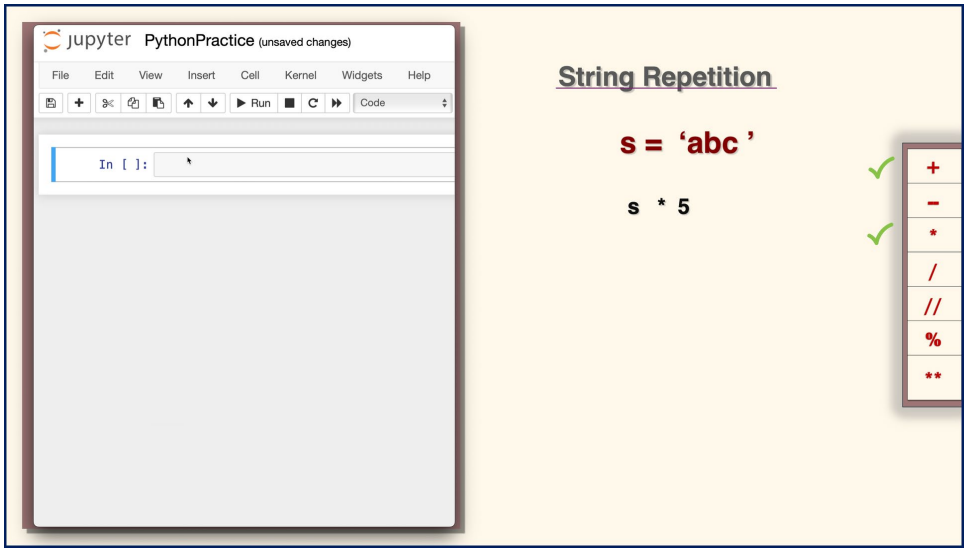

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [5]: s = 'a'
s * 5
Out[5]: 'aaaaa'
In [ ]:
String Repetition
s = ‘a’
‘aaaaa’ ⟵ s * 5
+
-
*
/
//
%
**

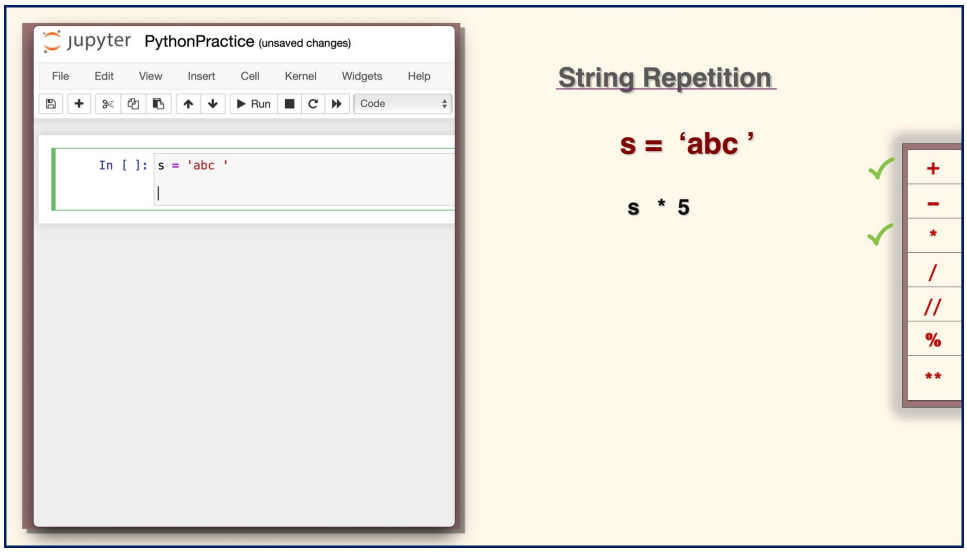

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [5]: s = 'a'
s * 5
Out[5]: 'aaaaa'
In [ ]: s = 'a'
String Repetition
s = 'a'
'aaaaa' ⟵ s * 5
+
-
*
/
//
%
**

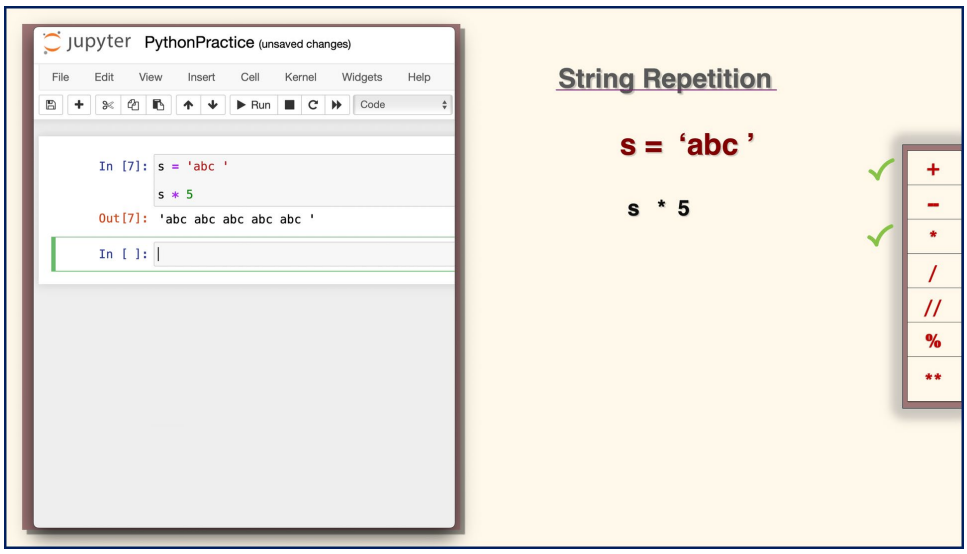

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [5]: s = 'a'
s * 5
Out[5]: 'aaaaa'
In [ ]: s = 'a'
s * 5.5
String Repetition
s = 'a'
'aaaaa' ⟵ s * 5
+
-
*
/
//
%
**

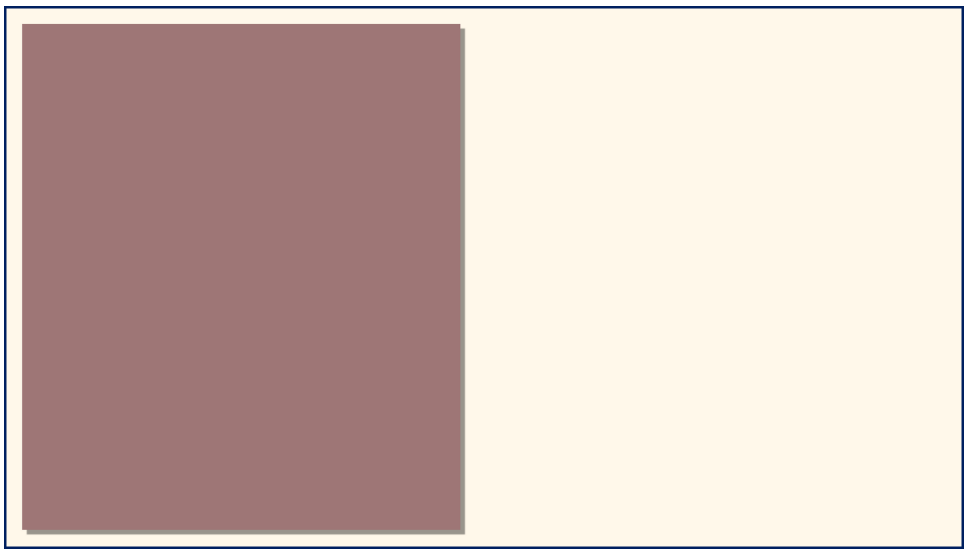

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [5]: s = 'a'
s * 5
Out[5]: 'aaaaa'
In [6]: s = 'a'
s * 5.5
TypeError
Cell In[6], line 3
1 s = 'a'
----> 3 s * 5.5
TypeError: can't multiply sequence by non-
In [ ]:
String Repetition
s = 'a'
'aaaaa' ← s * 5
+
-
*
/
//
%
**

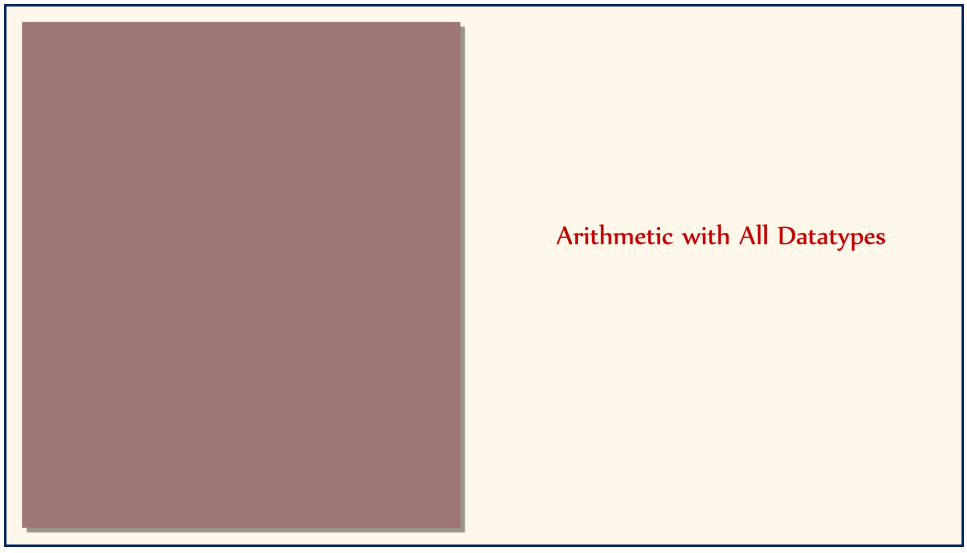

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [5]: s = 'a'
s = 5
Out[5]: 'aaaaa'
In [6]: s = 'a'
s = 5.5
TypeError Traceback (mo
Cell In[6], line 3
1 s = 'a'
----> 3 s = 5.5
TypeError: can't multiply sequence by non-int of type '
In [ ]:
String Repetition
s = ‘a’
‘aaaaa’ ⟵ s * 5
+
-
*
/
//
%
**

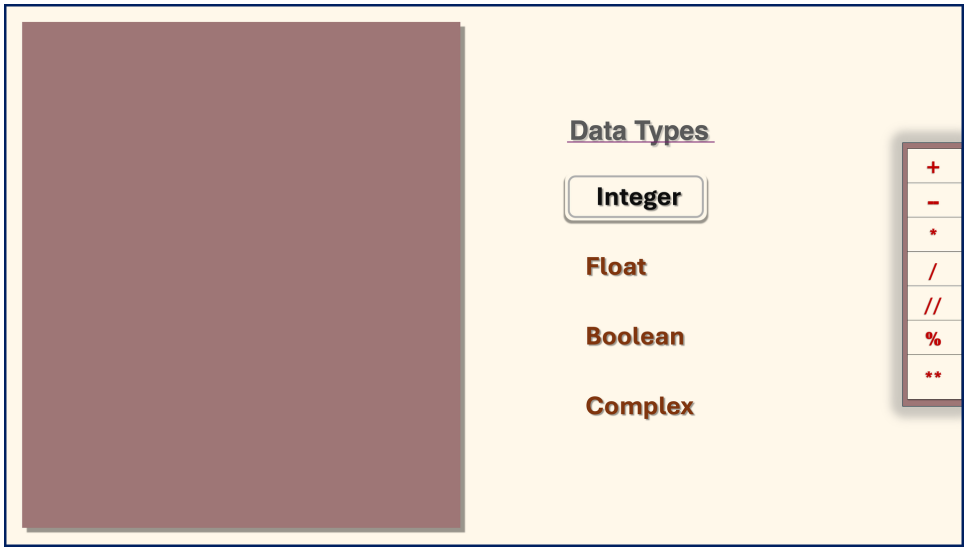

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]:
String Repetition
s = ‘abc ’
s * 5
+
-
*
/
//
%
**

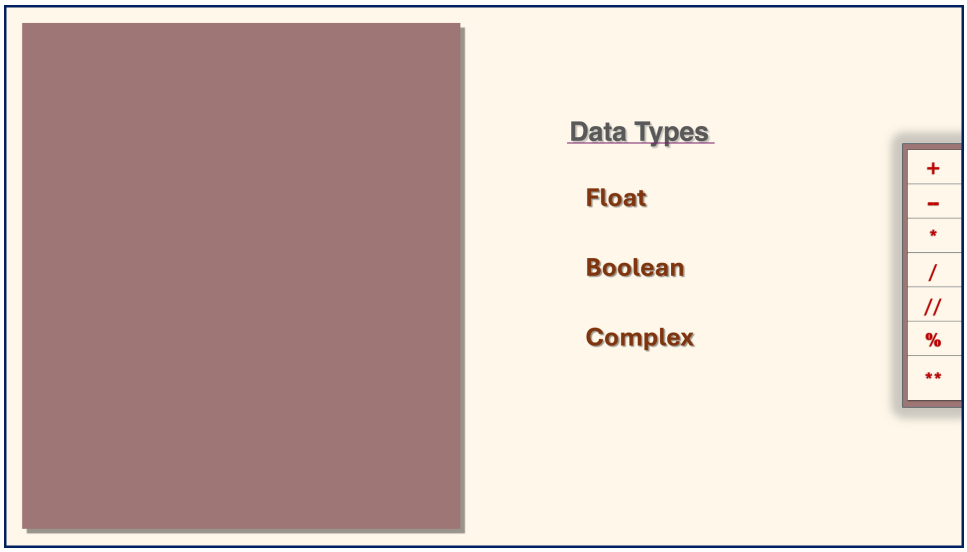

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [ ]: s = 'abc '
String Repetition
s = ‘abc ’
s * 5
+
-
*
/
//
%
**

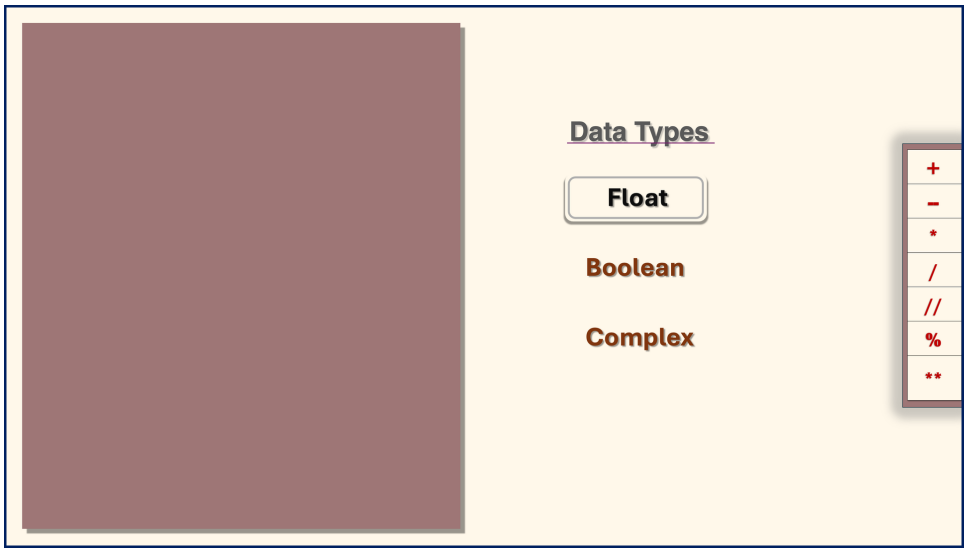

jupyter PythonPractice (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run Code
In [7]: s = 'abc '
s * 5
Out[7]: 'abc abc abc abc abc '
In [ ]:
String Repetition
s = ‘abc ’
s * 5
+
-
*
/
//
%
**

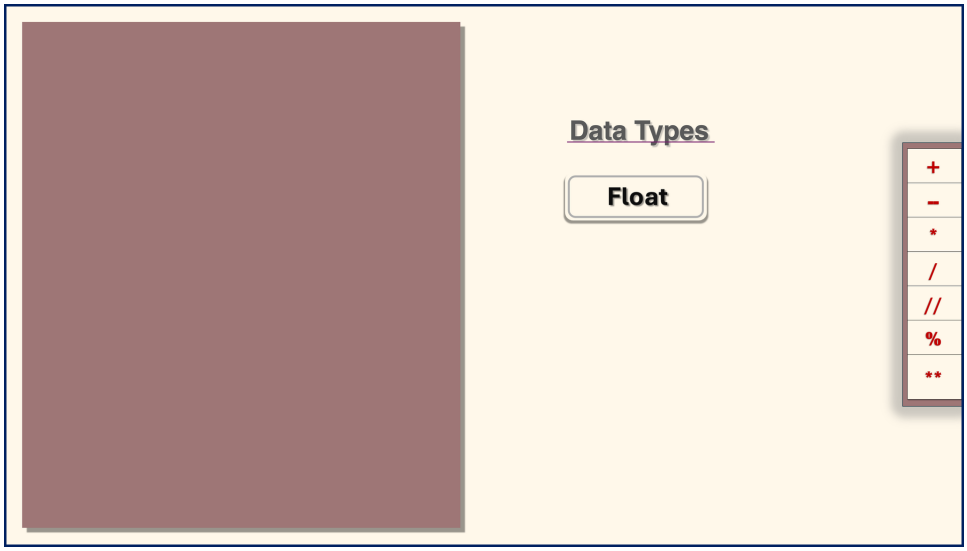


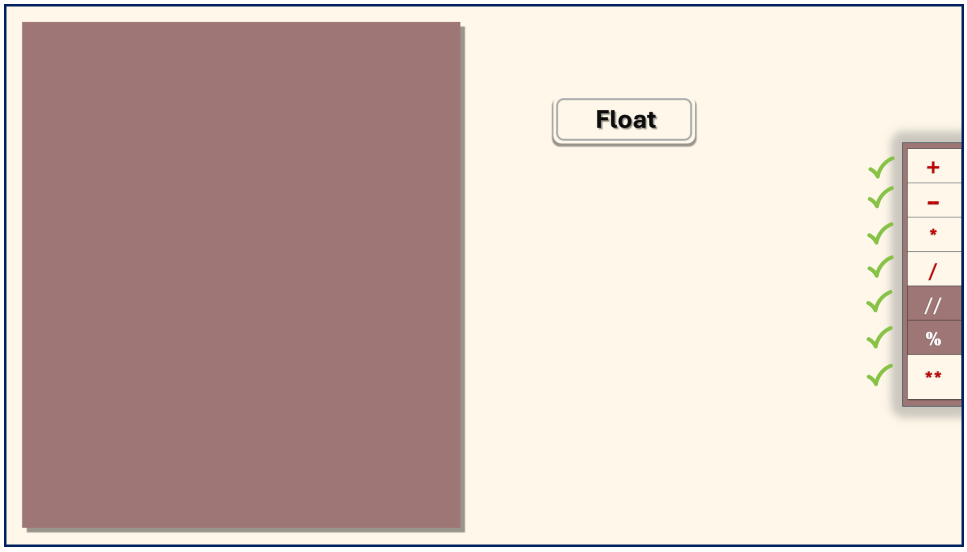

Arithmetic with All Datatypes

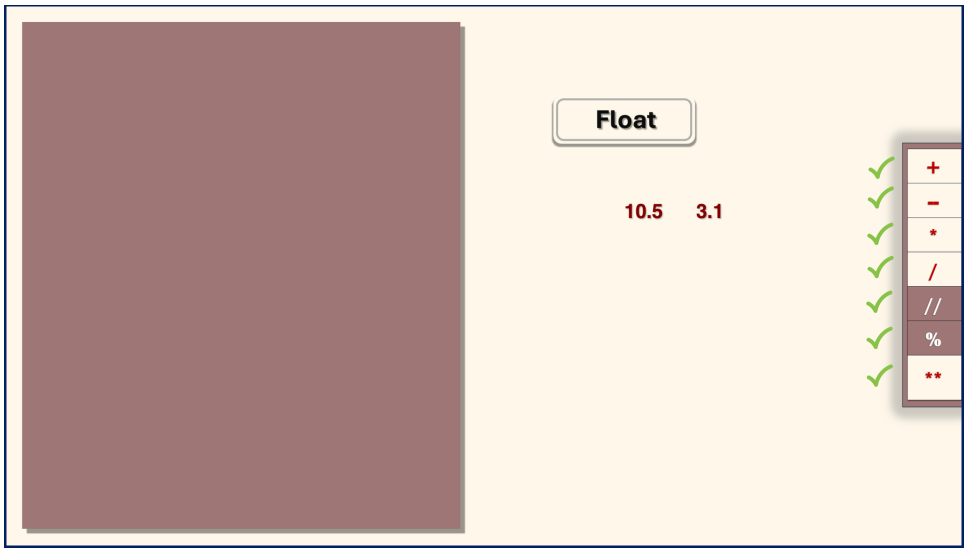

Data Types
Integer
Float
Boolean
Complex
+
-
*
/
//
%
**

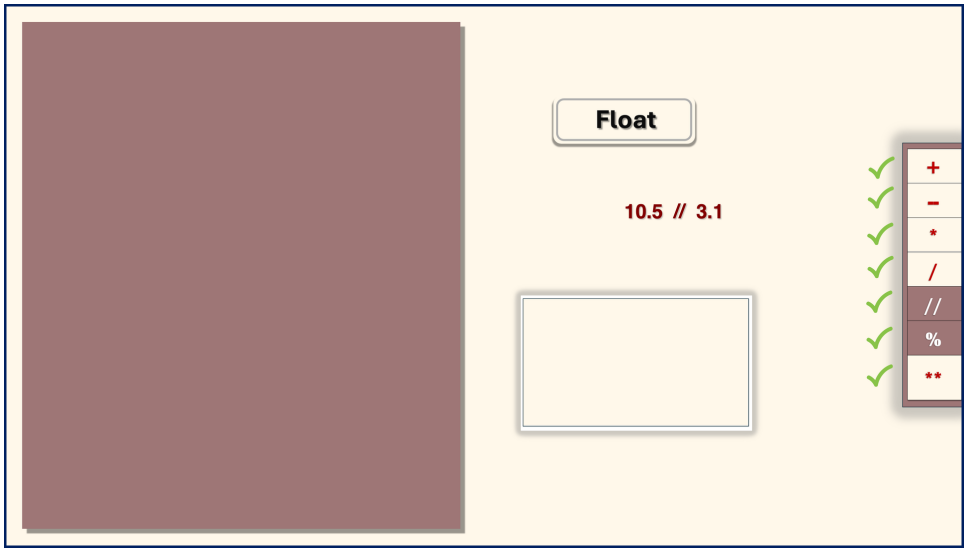

Data Types
Float
Boolean
Complex
+
-
*
/
//
%
**

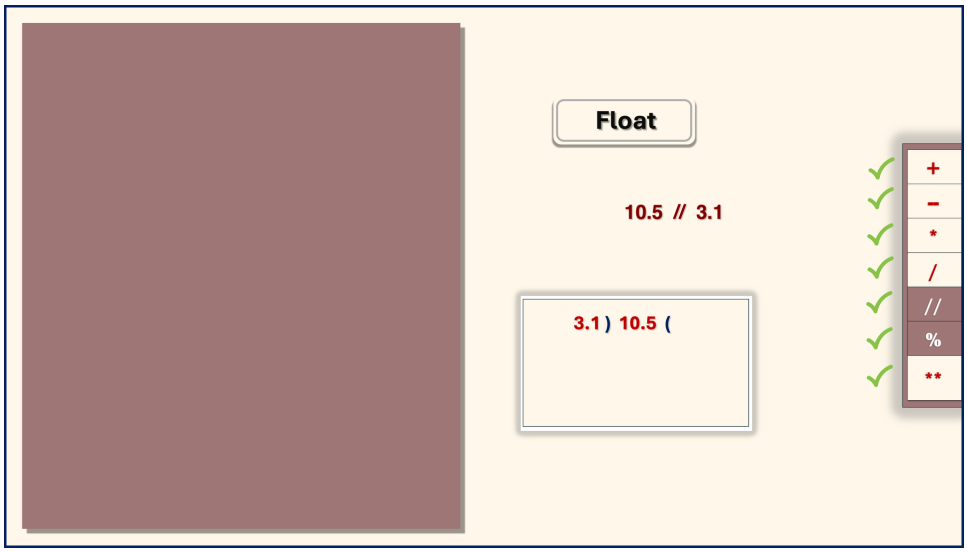

Data Types
Float
Boolean
Complex
+
-
*
/
//
%
**

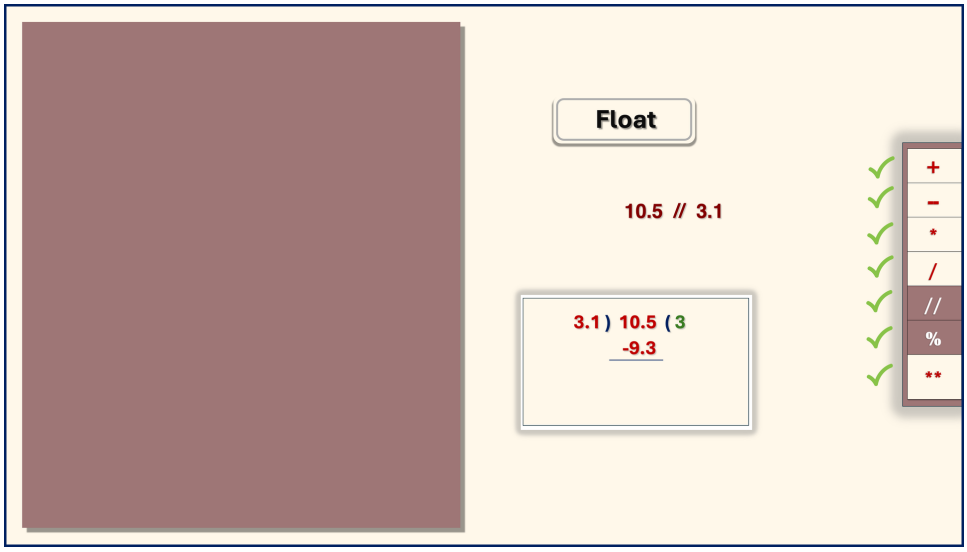

Data Types
Float
+
-
*
/
//
%
**

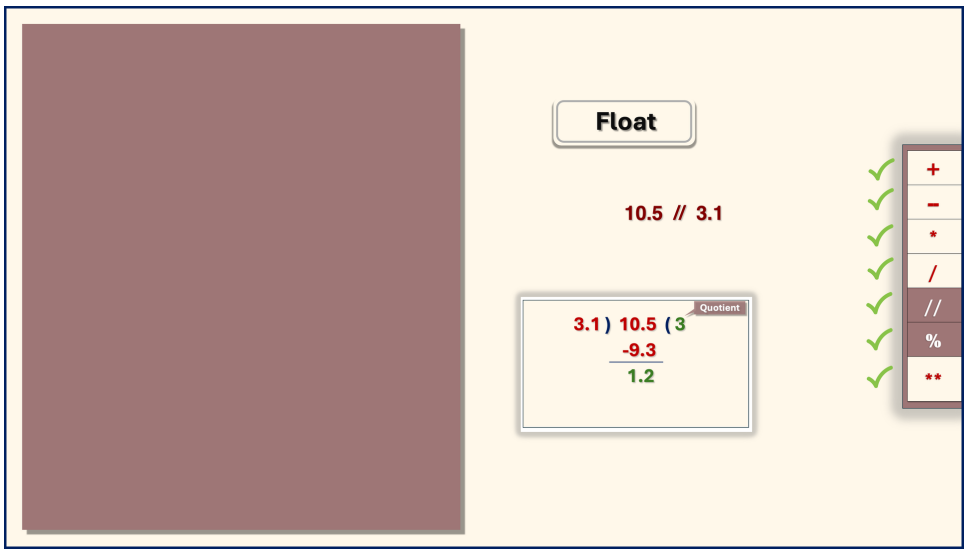

Float
+
-
*
/
//
%
**

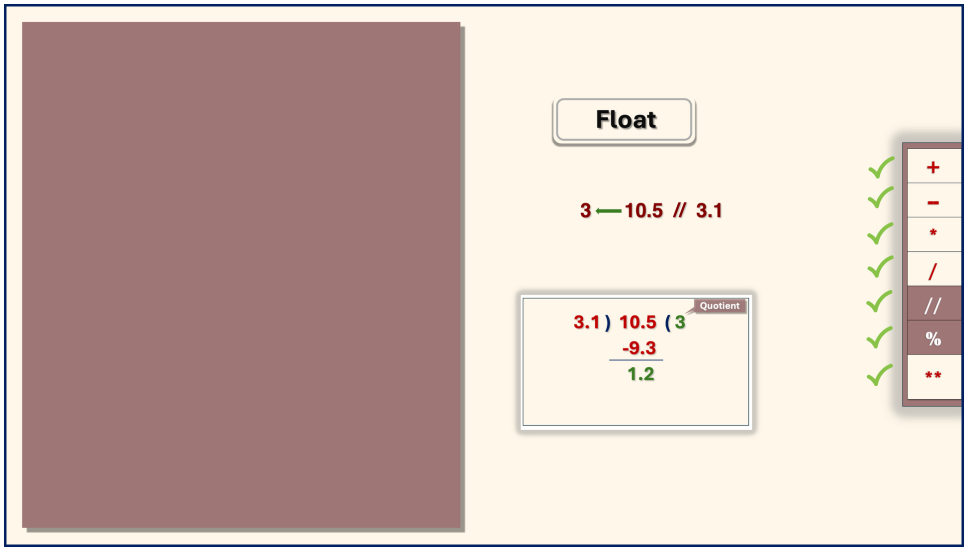

Float
10.5 3.1
+
-
*
/
//
%
**

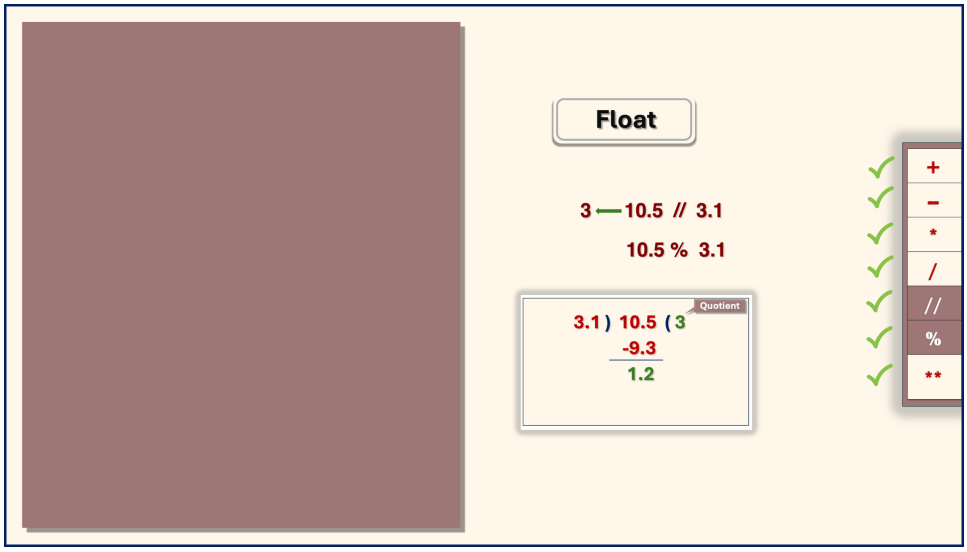

Float
10.5 // 3.1
+
-
*
/
//
%
**

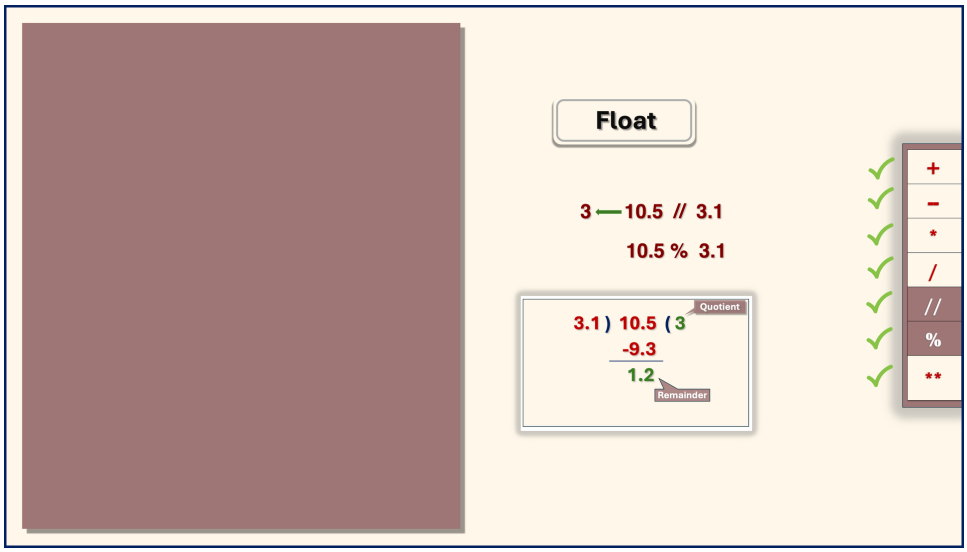

Float
10.5 // 3.1
3.1) 10.5 (
+
-
*
/
//
%
**

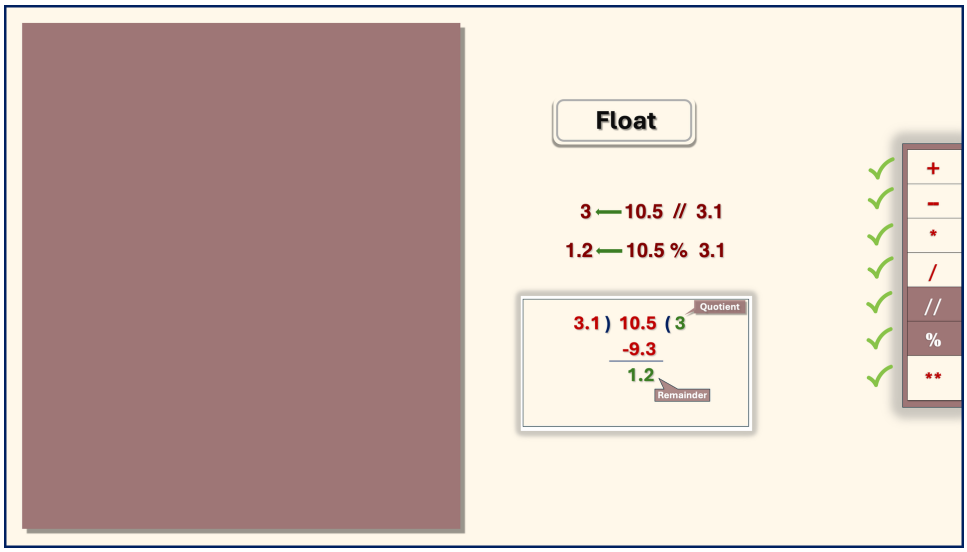

Float
10.5 // 3.1
3.1) 10.5 (3
-9.3
+
-
*
/
//
%
**

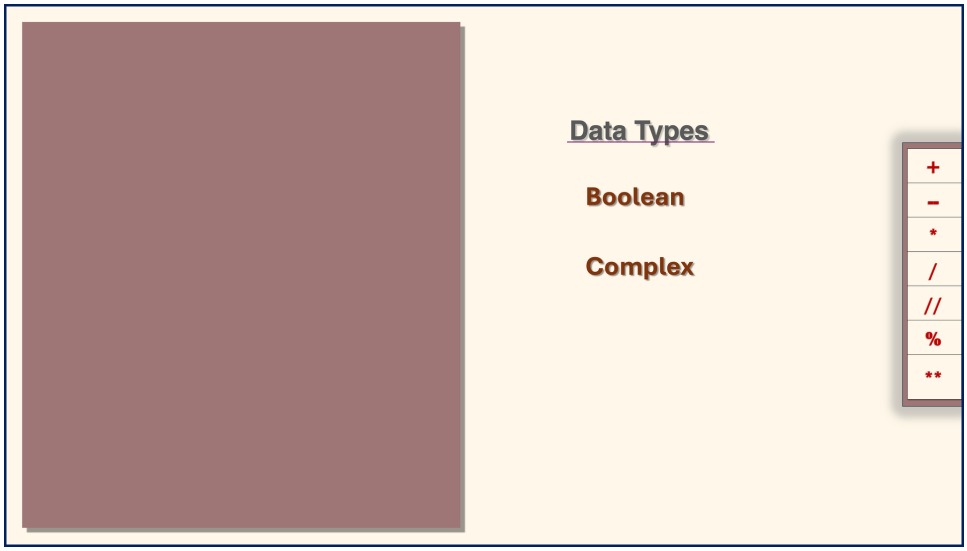

Float
10.5 // 3.1
Quotient
3.1) 10.5 (3
-9.3
1.2
+
-
*
/
//
%
**

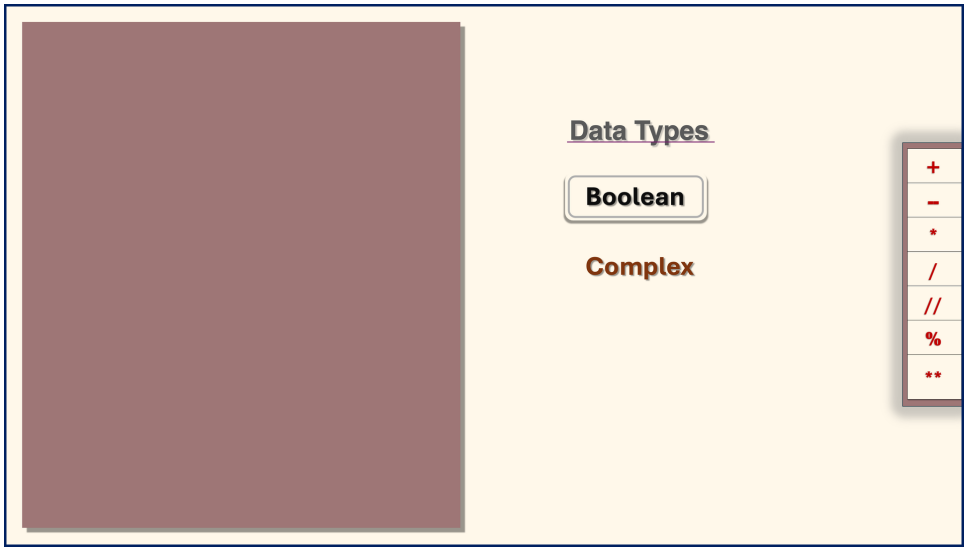

Float
3 ← 10.5 // 3.1
Quotient
3.1) 10.5 (3
-9.3
1.2
+
-
*
/
//
%
**

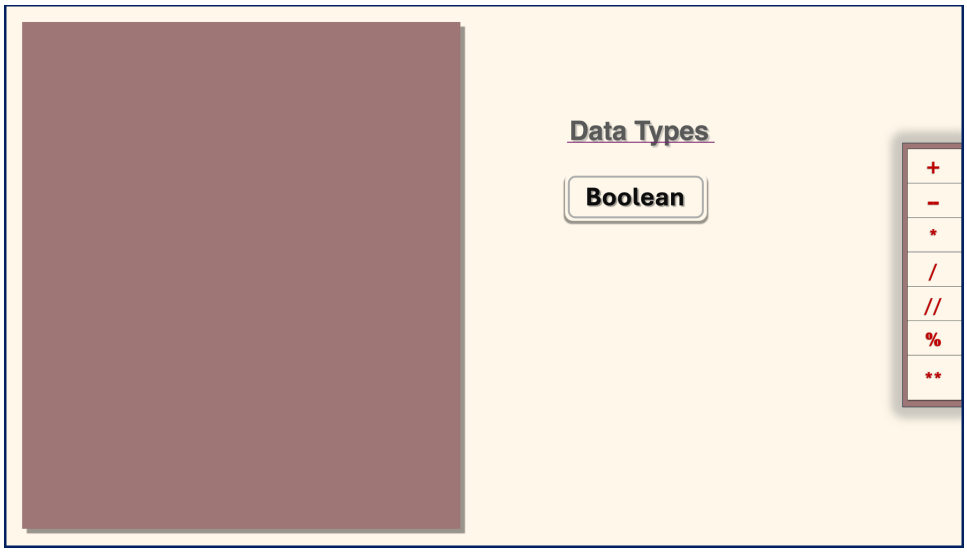

Float
3 ← 10.5 // 3.1
10.5 % 3.1
3.1) 10.5 (3
Quotient
-9.3
1.2
+
-
*
/
//
%
**

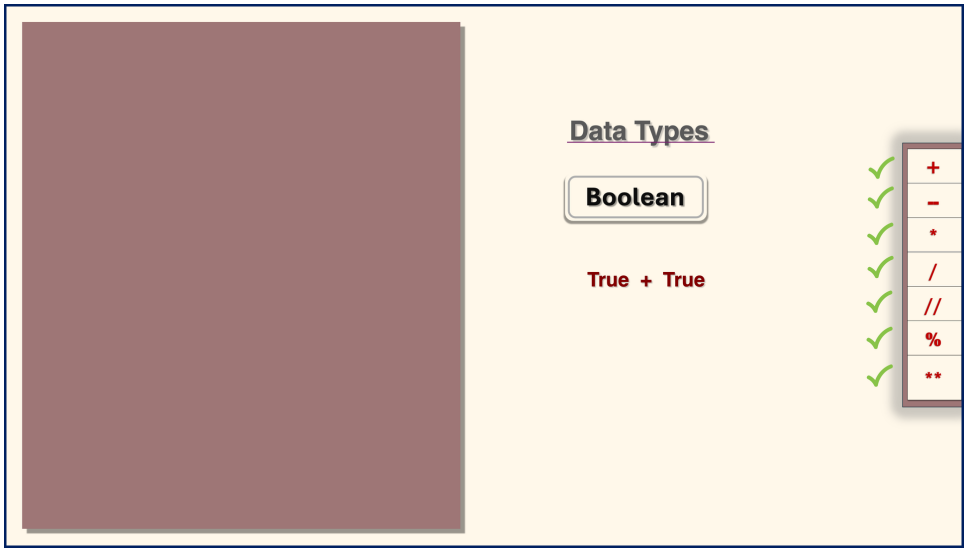

Float
3 ← 10.5 // 3.1
10.5 % 3.1
3.1) 10.5 (3
Quotient
-9.3
1.2
Remainder
+
-
*
/
//
%
**

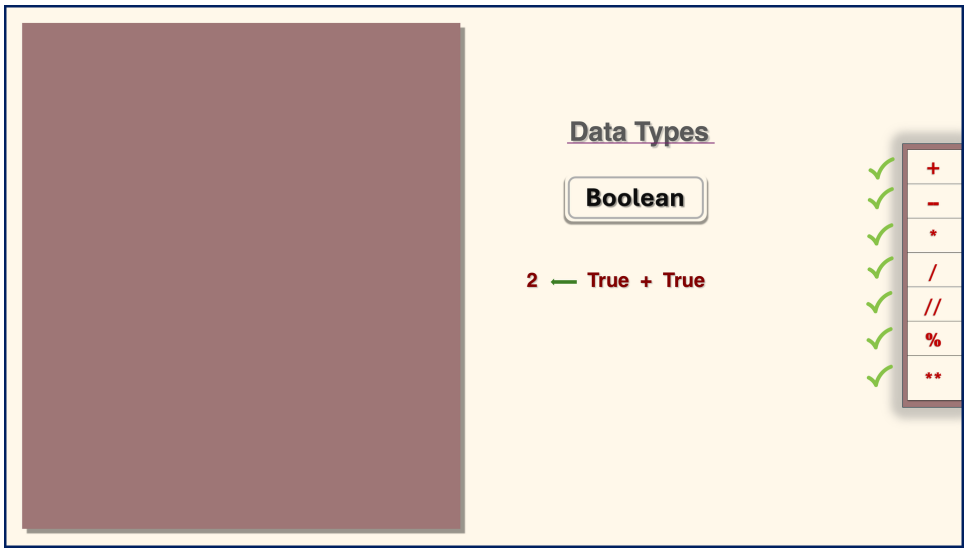

Float
3 ← 10.5 // 3.1
1.2 ← 10.5 % 3.1
Quotient
3.1) 10.5 (3
-9.3
1.2
Remainder
+
-
*
/
//
%
**

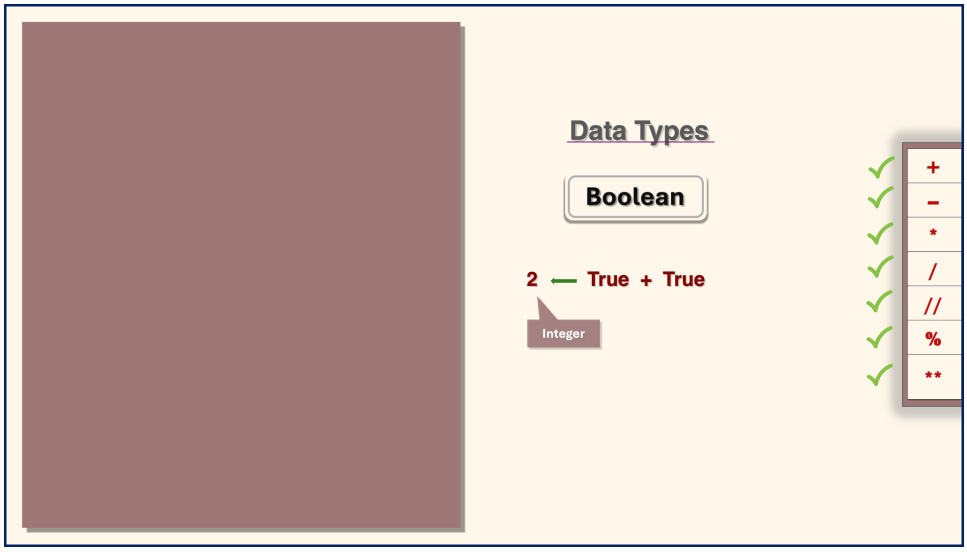

Data Types
Boolean
Complex
+
-
*
/
//
%
**

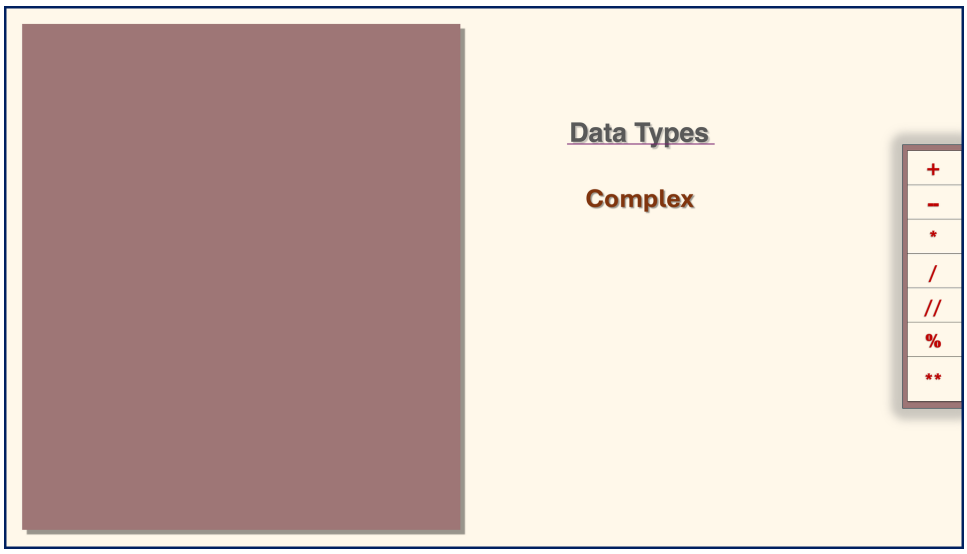

Data Types
Boolean
Complex
+
-
*
/
//
%
**

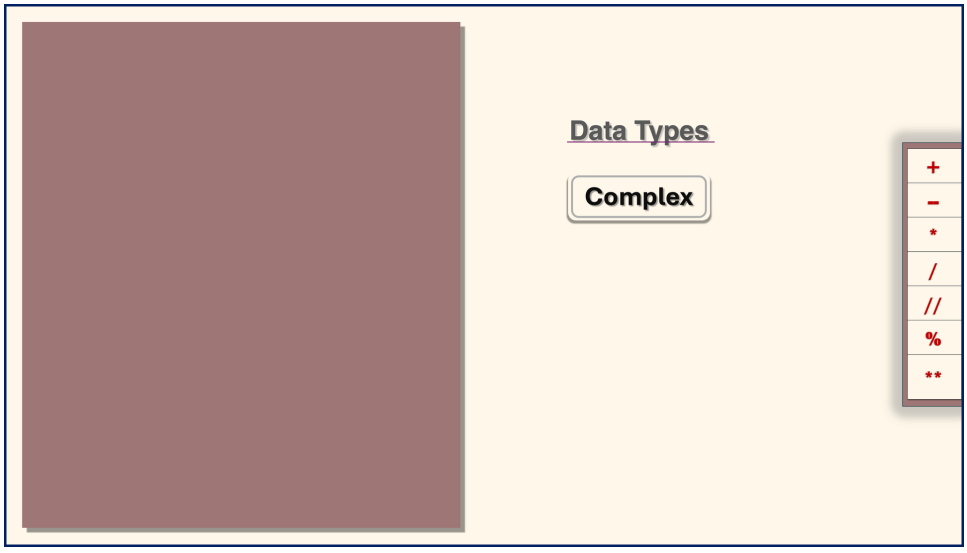

Data Types
Boolean
+
-
*
/
//
%
**

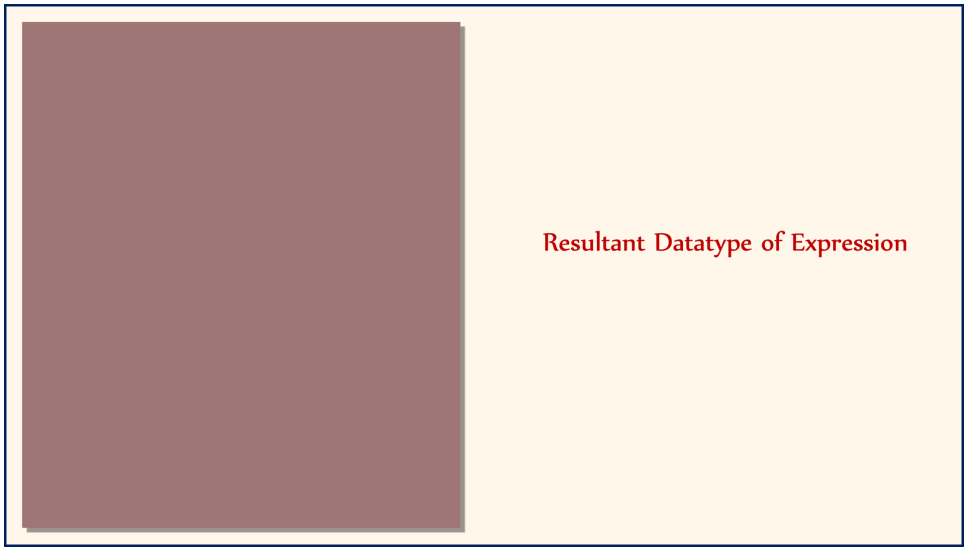

Data Types
Boolean
True + True
+
-
*
/
//
%
**

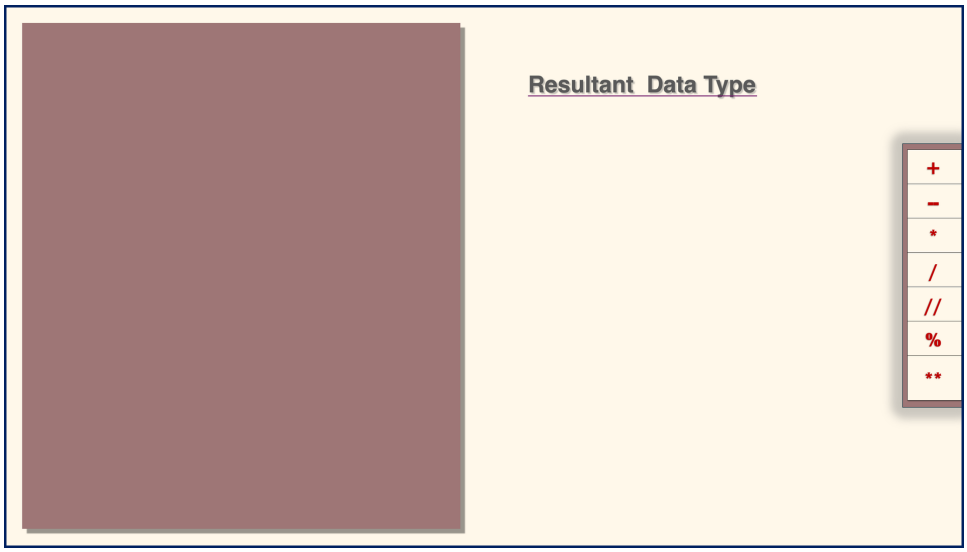

Data Types
Boolean
2 ⟵ True + True
+
-
*
/
//
%
**

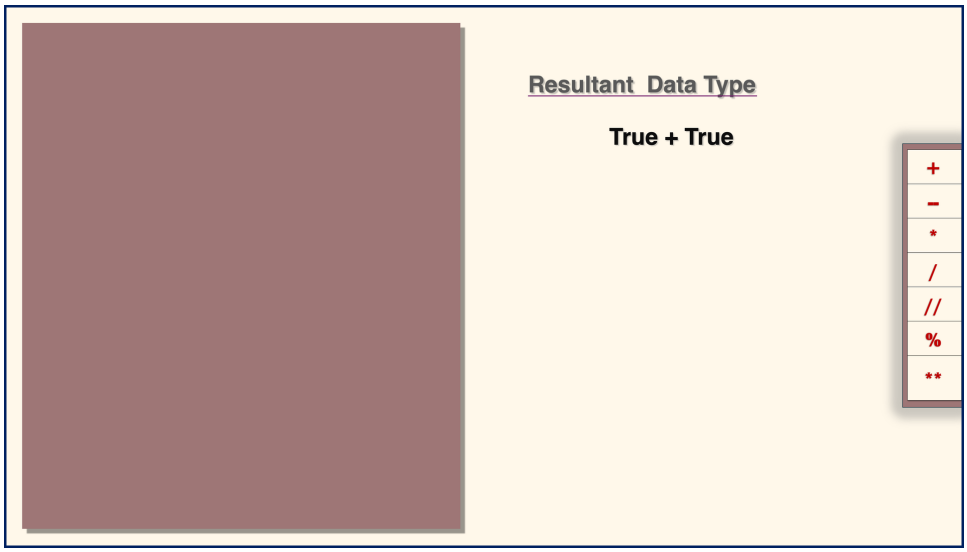

Data Types
Boolean
2 ⟵ True + True
Integer
+
-
*
/
//
%
**

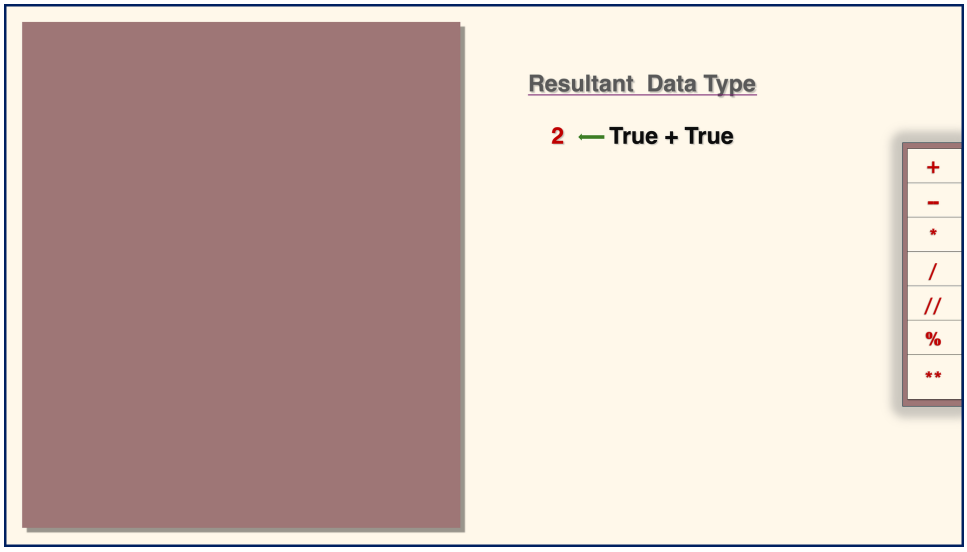

Data Types
Complex
+
-
*
/
//
%
**

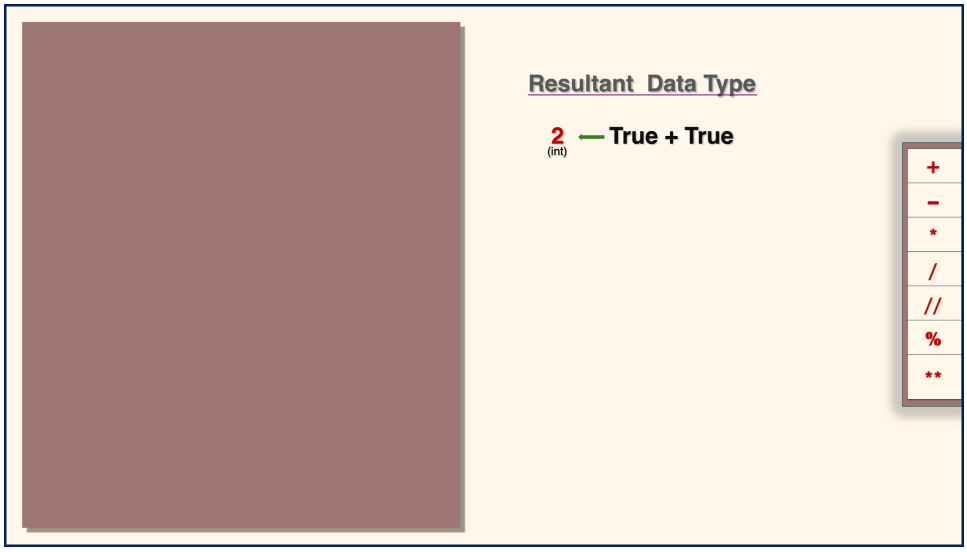

Data Types
Complex
+
-
*
/
//
%
**

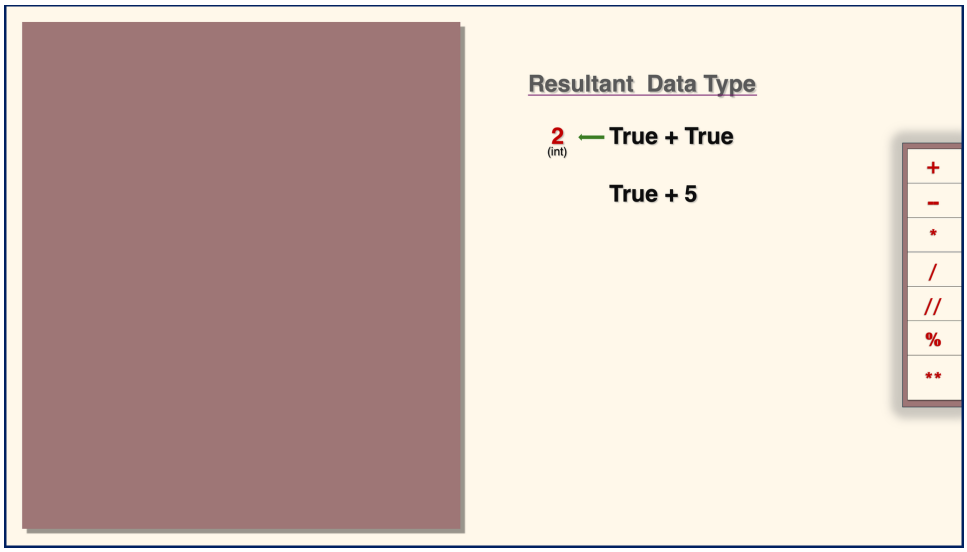

Resultant Datatype of Expression

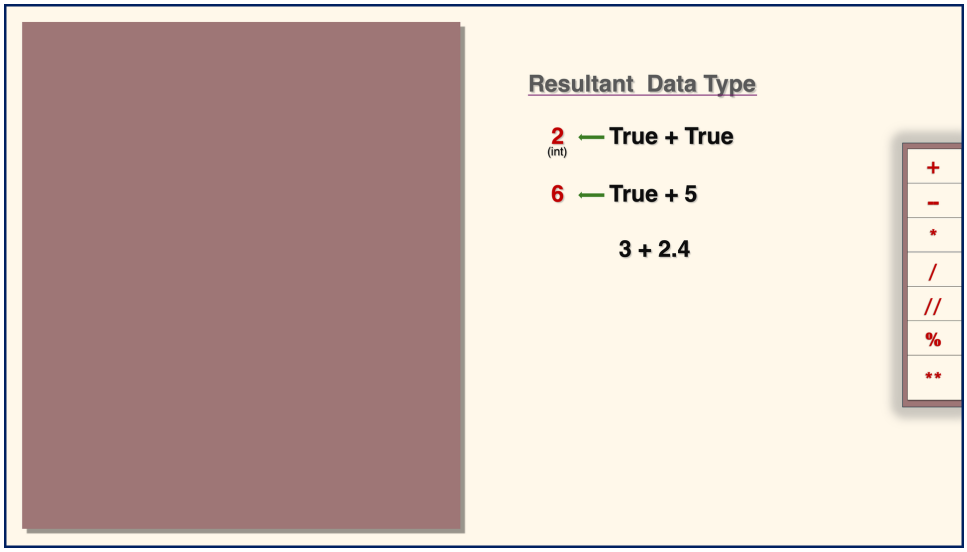

Resultant Data Type
+
-
*
/
//
%
**

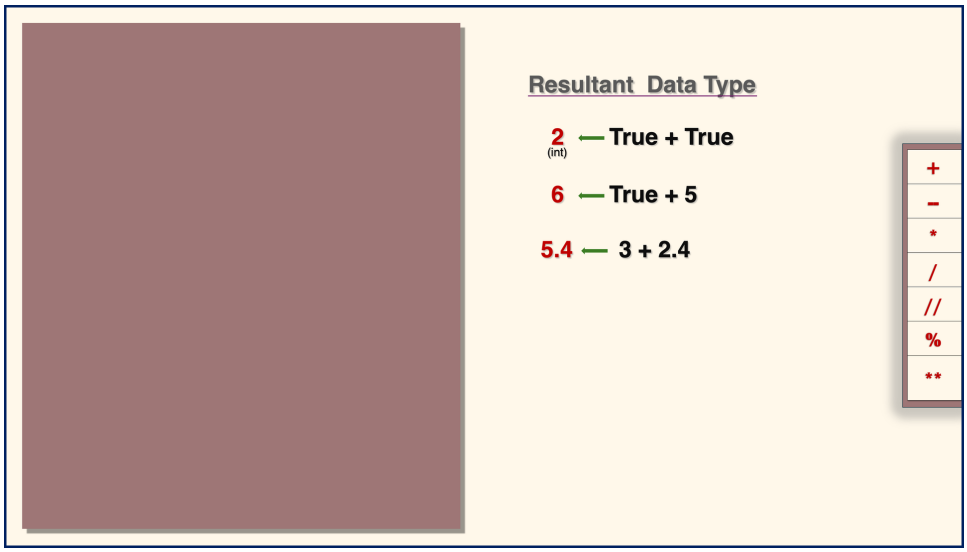

Resultant Data Type
True + True
+
-
*
/
//
%
**

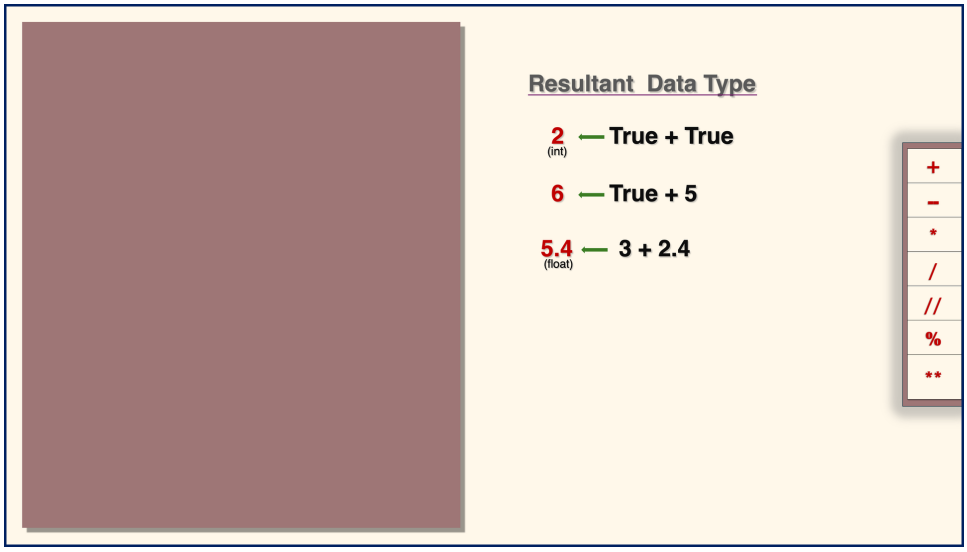

Resultant Data Type
2 ⟵ True + True
+
-
*
/
//
%
**

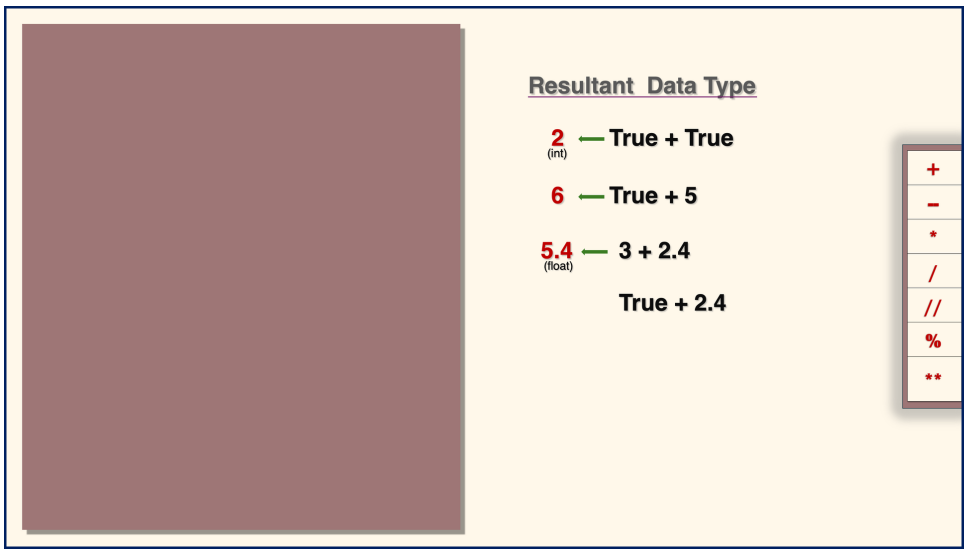

Resultant Data Type
2
(int)
True + True
+
-
*
/
//
%
**

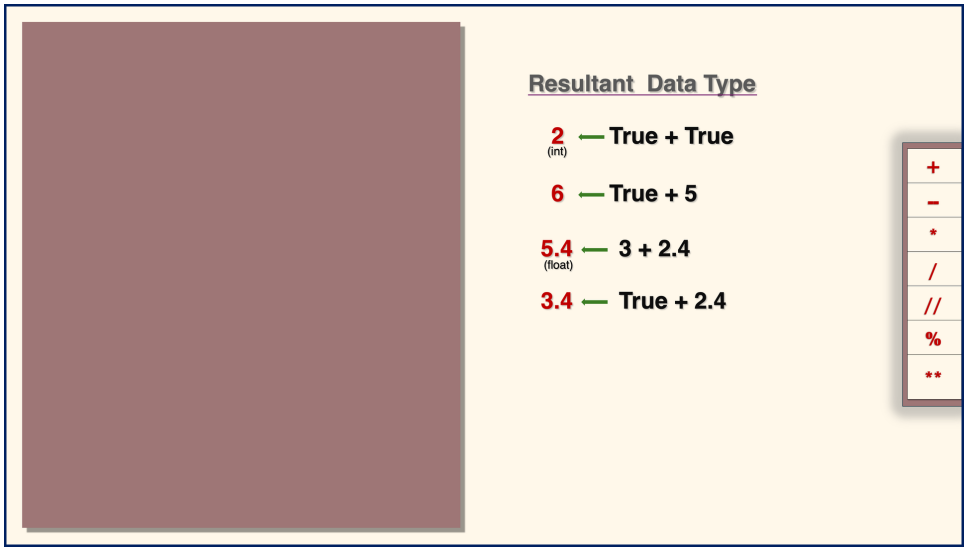

Resultant Data Type
2 (int) ← True + True
True + 5
+
-
*
/
//
%
**

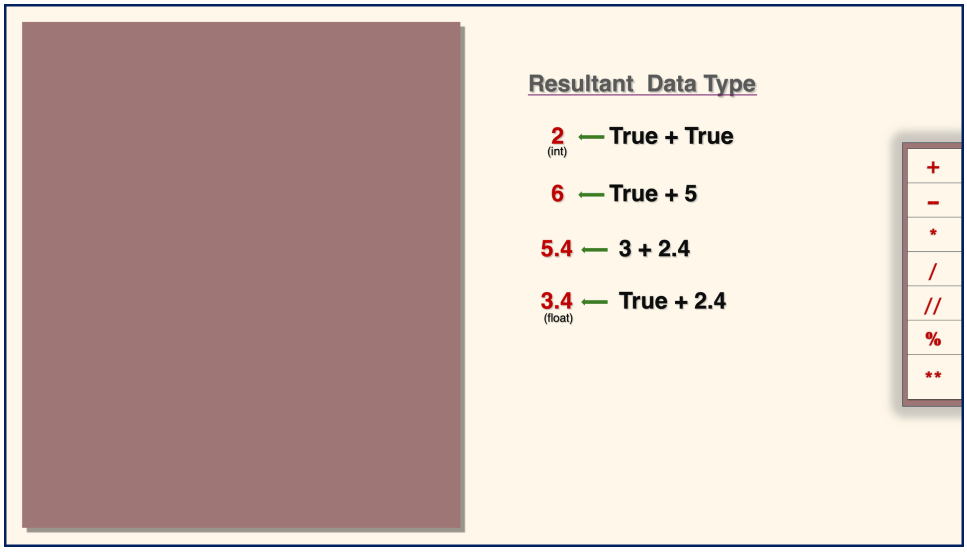

Resultant Data Type
2 (int) ← True + True
6 ← True + 5
3 + 2.4
+
-
*
/
//
%
**

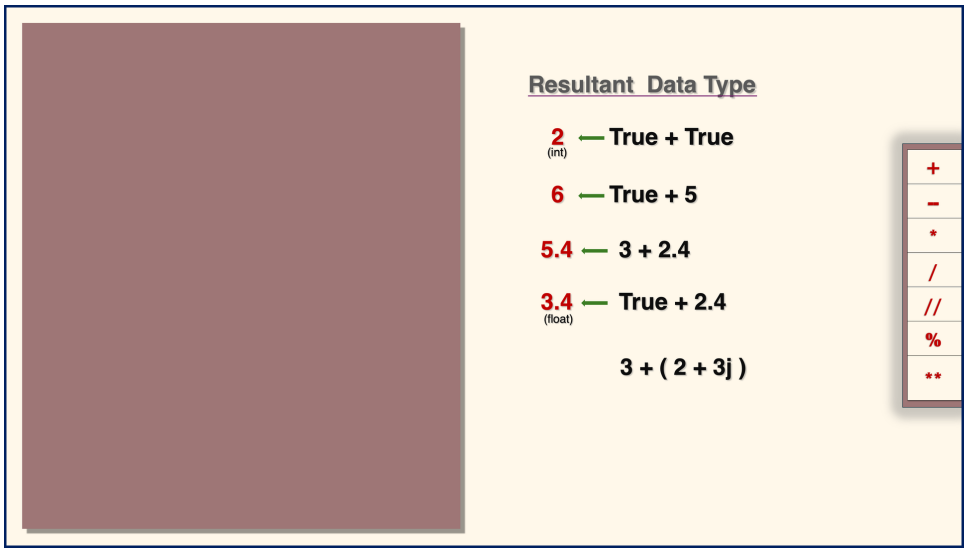

Resultant Data Type
2 (int) ← True + True
6 ← True + 5
5.4 ← 3 + 2.4
+
-
*
/
//
%
**

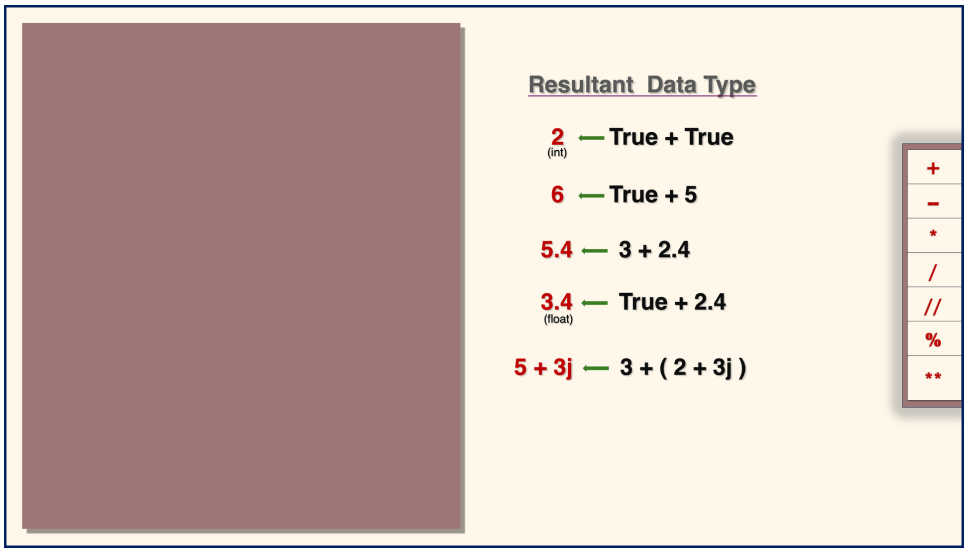

Resultant Data Type
2 (int) ← True + True
6 ← True + 5
5.4 (float) ← 3 + 2.4
+
-
*
/
//
%
**

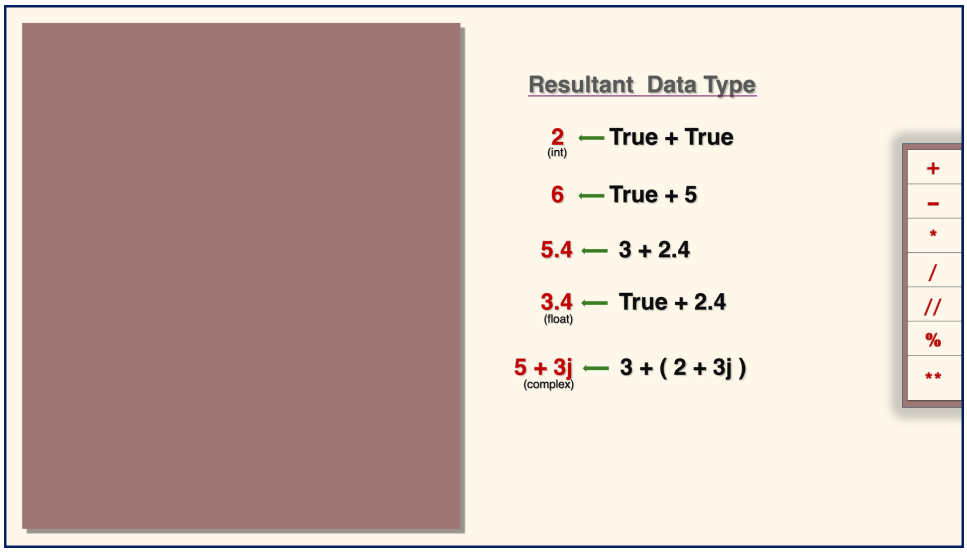

Resultant Data Type
2 (int) ← True + True
6 ← True + 5
5.4 (float) ← 3 + 2.4
True + 2.4
+
-
*
/
//
%
**

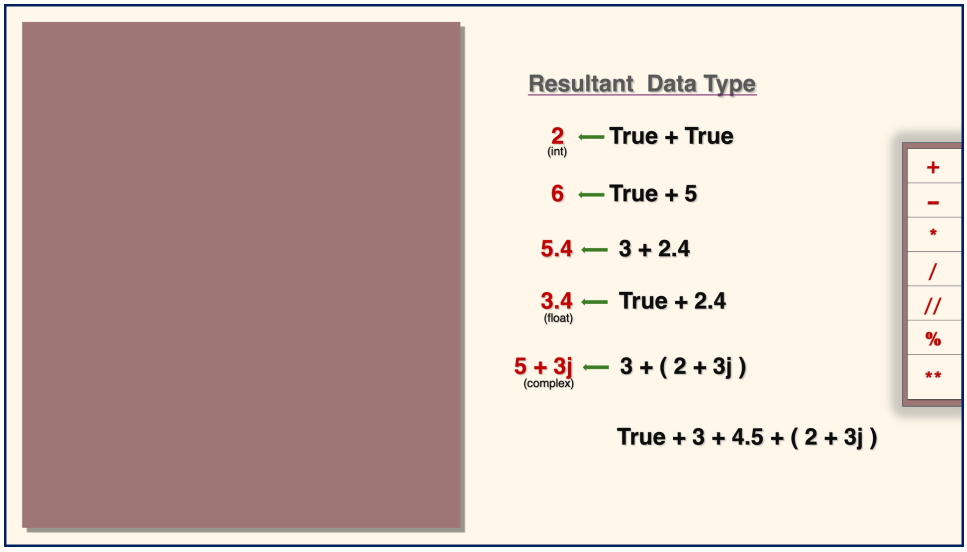

Resultant Data Type
2 (int) ← True + True
6 ← True + 5
5.4 (float) ← 3 + 2.4
3.4 ← True + 2.4
+
-
*
/
//
%
**

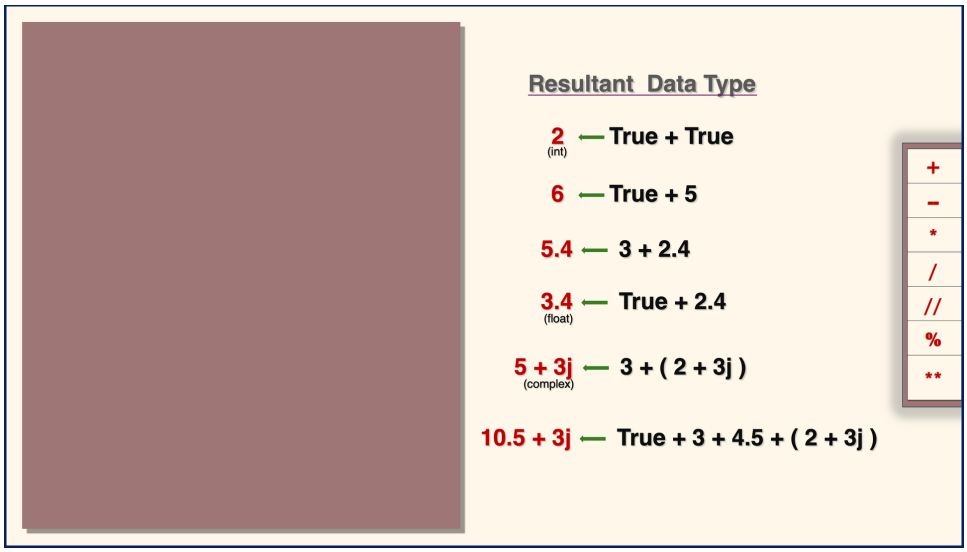

Resultant Data Type
2 (int) ← True + True
6 ← True + 5
5.4 ← 3 + 2.4
3.4 (float) ← True + 2.4
+
-
*
/
//
%
**

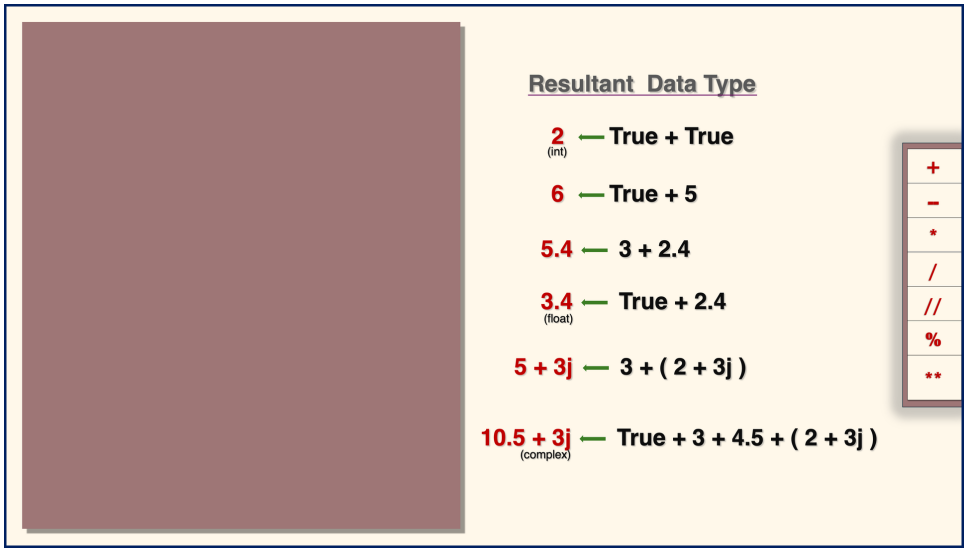

Resultant Data Type
2 (int) ← True + True
6 ← True + 5
5.4 ← 3 + 2.4
3.4 (float) ← True + 2.4
3 + (2 + 3j)
+
-
*
/
//
%
**

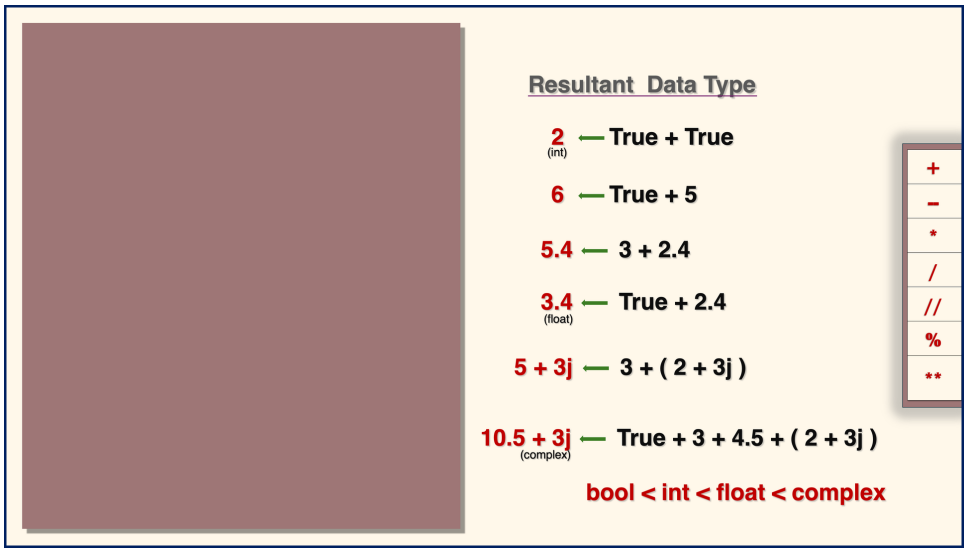

Resultant Data Type
2 (int) ⟵ True + True
6 ⟵ True + 5
5.4 ⟵ 3 + 2.4
3.4 (float) ⟵ True + 2.4
5 + 3j ⟵ 3 + (2 + 3j)
+
-
*
/
//
%
**


Resultant Data Type
2 (int) ⟵ True + True
6 ⟵ True + 5
5.4 ⟵ 3 + 2.4
3.4 (float) ⟵ True + 2.4
5 + 3j (complex) ⟵ 3 + (2 + 3j)
+
-
*
/
//
%
**


Resultant Data Type
2 (int) ← True + True
6 ← True + 5
5.4 ← 3 + 2.4
3.4 (float) ← True + 2.4
5 + 3j (complex) ← 3 + (2 + 3j)
True + 3 + 4.5 + (2 + 3j)
+
-
*
/
//
%
**


Resultant Data Type
2 (int) ⟵ True + True
6 ⟵ True + 5
5.4 ⟵ 3 + 2.4
3.4 (float) ⟵ True + 2.4
5 + 3j (complex) ⟵ 3 + (2 + 3j)
10.5 + 3j ⟵ True + 3 + 4.5 + (2 + 3j)
+
-
*
/
//
%
**


Resultant Data Type
2 (int) ⟵ True + True
6 ⟵ True + 5
5.4 ⟵ 3 + 2.4
3.4 (float) ⟵ True + 2.4
5 + 3j ⟵ 3 + (2 + 3j)
10.5 + 3j (complex) ⟵ True + 3 + 4.5 + (2 + 3j)
+
-
*
/
//
%
**


Resultant Data Type
2 (int) ⟵ True + True
6 ⟵ True + 5
5.4 ⟵ 3 + 2.4
3.4 (float) ⟵ True + 2.4
5 + 3j ⟵ 3 + (2 + 3j)
10.5 + 3j (complex) ⟵ True + 3 + 4.5 + (2 + 3j)
bool < int < float < complex
+
-
*
/
//
%
**